

YABA COLLEGE OF TECHNOLOGY

SCHOOL OF TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE

**DESIGN AND IMPLEMENTATION OF A DIGITAL PROJECT SUPERVISION
AND PROGRESS TRACKING SYSTEM USING A RULE-BASED MILESTONE
PROGRESS TRACKING ALGORITHM (A CASE STUDY OF YABA COLLEGE
OF TECHNOLOGY)**

BY

[STUDENT FULL NAME]

MATRICULATION NUMBER: [MATRIC NUMBER]

**BEING A PROJECT SUBMITTED TO THE DEPARTMENT OF COMPUTER
SCIENCE, SCHOOL OF TECHNOLOGY, YABA COLLEGE OF
TECHNOLOGY, IN PARTIAL FULFILMENT OF THE REQUIREMENTS FOR
THE AWARD OF HIGHER NATIONAL DIPLOMA (HND) IN COMPUTER
SCIENCE**

OCTOBER 2026

TABLE OF CONTENTS

CHAPTER ONE	3
1.1 Background of the Study	3
1.2 Statement of the Problem	5
1.3 Aim of the Study	5
1.4 Objectives of the Study	5
1.5 Research Questions & Hypothesis	6
1.6 Significance / Justification	6
1.7 Scope of the Study	7
1.8 Methodology Overview	7
1.9 Limitations of the Study	8
1.10 Definition of Terms	8
CHAPTER TWO	10
2.0 Introduction	10
2.1 Conceptual Framework	10
2.2 Theoretical Framework	11
2.3 Empirical Framework	11
2.4 Review of Related Work	12
2.5 Gap in Literature	21
CHAPTER THREE	22
3.0 Introduction	22
3.1 System Development Methodology	22
3.2 Analysis of the Existing System	23
3.3 Analysis of the Proposed System	24
3.4 System Requirements	25
3.5 System Architecture	26
3.6 Database and Data Design	27
3.7 Rule-based Milestone Progress and Project Health Algorithms	29
3.8 Security and Access Control	32
3.9 Development and Deployment Tools	33
CHAPTER FOUR	34
4.0 Introduction	34
4.1 Implementation Overview	34
4.2 User Interface Implementation	35
4.3 Testing and Results	48
4.4 Implementation and System Behaviour	50
4.5 User Guide	52
4.6 Chapter Summary	54
REFERENCES	55

CHAPTER ONE

INTRODUCTION

1.1 Background of the Study

Project supervision is the process by which an academic supervisor guides a student through the stages of conceiving, developing and documenting a final year or Higher National Diploma (HND) project. It is one of the most demanding responsibilities placed on academic staff, since a single supervisor may be required to guide dozens of students at once, while each student progresses through the same broad stages of proposal writing, literature review, design, implementation and testing at a different pace. In most Nigerian polytechnics, this process is still conducted almost entirely on paper: students record supervision meetings in handwritten logbooks, corrections are made directly on printed chapters, and appointments are scheduled informally by knocking on a supervisor's office door or sending a text message.

This manual arrangement creates several recurring difficulties. Logbooks are easily misplaced by students between semesters, and a supervisor who is away from the department for even a short period has no way of knowing how far a student has progressed since the last physical meeting. Feedback on a submitted chapter can take days or weeks to reach a student, not because the supervisor is unwilling to help, but because there is no channel through which comments can be delivered outside a face-to-face encounter. Where a department runs several hundred HND projects in a single session, the Head of Department and the project coordinator have no consolidated view of which students are on schedule, which have stalled, and which supervisors are overloaded; this information exists only in scattered, unsynchronised paper records held by individual supervisors.

Yaba College of Technology, popularly known as YABATECH, is a federal tertiary institution located in Yaba, Lagos State, and is widely regarded as the first higher educational institution established in Nigeria. Its roots lie in Yaba Higher College, which was founded in 1932 to provide tertiary education, mostly in vocational and teaching subjects. Following the recommendation of the Elliot Commission on higher education in West Africa, the Higher College was closed and its students were moved in 1948 to form the nucleus of the University College, Ibadan, while the Yaba Technical Institute, which

started day and evening classes in October 1947, took over its activities and premises (Yaba College of Technology, n.d.).

The Institute was later renamed Yaba College of Technology, and in 1969 a Federal Government decree established it as an autonomous institution. Under the Federal Polytechnics Act of 1979, it was renamed Federal Polytechnic, Yaba, but it reverted to its original name in 1980; until 1979 it was the only institution in Nigeria offering Higher Diploma courses. Today the College operates under the regulation of the National Board for Technical Education (NBTE) and offers National Diploma (ND), Higher National Diploma (HND) and Post-HND programmes in science, technology, commerce and related fields. It was also the first higher institution in Nigeria to establish a full-fledged Centre for Entrepreneurship Development (Yaba College of Technology, n.d.). The Department of Computer Science, whose HND project supervision is the subject of this study, operates within this institutional setting, which makes the College a representative and appropriate case study for examining the challenges of project supervision in Nigerian polytechnics.

At Yaba College of Technology, these problems are visible every session in the Department of Computer Science. Final-year HND students are required to complete a project as a condition for graduation, and the department assigns supervisors from a relatively small pool of academic staff to a much larger number of students. Supervision meetings are held physically, in the supervisor's office, and the only record of what was discussed, and of the corrections made, is whatever the student manages to write in a personal notebook. When a student wishes to confirm the date of the last correction made to a chapter, or a supervisor wishes to review how many students have completed their literature review, no digital record exists to consult. This lack of a structured, time-stamped record of supervision activity makes it difficult to resolve disagreements about whether feedback was given, and makes it almost impossible for departmental administration to track overall project progress across cohorts.

Advances in web-based information systems make it possible to supplement paper-based supervision with a shared digital platform accessible from an internet-connected device. The platform can keep timestamped topic decisions, submission records, chapter reviews and feedback, while applying explicit rules to represent project progress. In the implementation developed for this study, the five chapter stages have equal weights of 20 per cent each; a stage contributes its weight only when the assigned supervisor approves

the current chapter submission. Topic approval is the entry condition for chapter work, while final project approval is recorded separately after every chapter is approved. A further rule set classifies project health using activity, submission recency and the scheduled defence date. Together, these measures provide a transparent basis for monitoring supervision in the Department of Computer Science, Yaba College of Technology.

1.2 Statement of the Problem

The Department of Computer Science at Yaba College of Technology currently has no digital system for supervising and tracking HND research projects, so supervision records are kept on paper by individual supervisors and students, feedback on submitted work is delayed until the next physical meeting, and neither the project coordinator nor the Head of Department has a reliable, up-to-date view of how far any given student has progressed. This study therefore designs and implements a digital project supervision and progress tracking system, driven by a rule-based milestone progress tracking algorithm, to replace this manual arrangement with a structured, real-time record of supervisor-student communication and project progress.

1.3 Aim of the Study

The aim of this study is to design and develop a digital project supervision and progress tracking system that applies a rule-based milestone progress tracking algorithm to automate the monitoring of student project progress and streamline supervisor-student communication at Yaba College of Technology.

1.4 Objectives of the Study

To achieve the stated aim, the study pursues the following specific objectives:

- i. To investigate the existing manual project supervision process in the Department of Computer Science, Yaba College of Technology, and identify its specific shortcomings.
- ii. To design the system architecture, database structure and a rule-based milestone progress tracking algorithm capable of computing a student's project progress from defined supervision milestones.

- iii. To develop a web-based platform that implements the designed architecture and algorithm, incorporating separate interfaces for students, supervisors and the Head of Department for departmental oversight.
- iv. To implement milestone weighting, automatic progress-percentage computation, explicit topic and final-approval conditions, and event notifications together with rule-based project-health indicators.
- v. To verify the developed system against the study objectives through automated regression checks and functional acceptance testing of the deployed role-based workflows.

1.5 Research Questions & Hypothesis

The following research questions guided the study:

- i. What are the specific shortcomings of the existing manual project supervision process in the Department of Computer Science, Yaba College of Technology?
- ii. What milestones, rules and weighting scheme should a rule-based algorithm use to compute an accurate and meaningful progress percentage for an HND project?
- iii. What features must a digital project supervision and progress tracking system provide to meet the needs of students, supervisors and departmental project oversight at Yaba College of Technology?
- iv. How does the developed system, evaluated against the existing manual process, affect the timeliness of supervisor feedback and the visibility of student progress?

Hypothesis: The implementation of a digital project supervision and progress tracking system driven by a rule-based milestone progress tracking algorithm will significantly reduce the average time taken for a supervisor's feedback to reach a student, and will improve the accuracy with which project progress can be reported, compared with the existing manual process.

1.6 Significance / Justification

This study is significant to several groups of stakeholders within Yaba College of Technology. Students benefit from a shared platform for selecting a departmental supervisor, proposing a topic, working with registered partners, submitting chapters and

receiving feedback. Chapter states and the progress percentage distinguish approved work from outstanding work, while activity records explain topic, review and extension decisions. Supervisors benefit from matriculation-number identification, consolidated project records, document review, grading controls and clear indicators of inactivity or deadline pressure. The Head of Department gains department-wide project and supervisor summaries for follow-up and workload discussions. The institution gains a timestamped supervision record that can support accountability and review. The study contributes a documented implementation of transparent progress rules and coordinated academic supervision in a Nigerian polytechnic setting.

1.7 Scope of the Study

This study focuses on a web-based digital project supervision and progress tracking system for HND students in the Department of Computer Science, Yaba College of Technology, with the 2024/2025 academic session retained as the reference case. The implemented workflow covers registration, student matriculation-number login, departmental supervisor selection, topic submission and review, project partners, chapter submission and revision, feedback, grades, activity records, deadline-extension requests, final approval and defence-date tracking. Five equally weighted chapter milestones provide the progress percentage, while separate activity and schedule rules produce project-health indicators. The Head of Department interface supports departmental monitoring. The application also accepts ND matriculation formats and provides other department options; the report's case-study focus remains Computer Science HND supervision. Plagiarism detection, payment processing, external-examiner allocation, defence-panel management, defence results and certification are outside the implemented scope.

1.8 Methodology Overview

The study combined workflow analysis, system design, implementation and verification. A sequential development lifecycle organised the initial requirements, architecture, data design, implementation, testing and deployment; later requirement corrections were incorporated through targeted revisions. The application uses Next.js 14 and TypeScript, Firebase Authentication and Cloud Firestore, private Supabase Storage, and Vercel hosting. Verification combined automated handler and session tests, public-page browser checks and a completed functional acceptance exercise on the deployed

application. Chapters Three and Four present the design, operational rules, implementation and results.

1.9 Limitations of the Study

- i. Access to some existing supervision and project records at Yaba College of Technology was restricted for confidentiality reasons, which limited the depth of analysis of past project outcomes that could be carried out during the existing-system study.
- ii. The study was limited to the Department of Computer Science at a single institution, so the milestone weighting scheme and findings may not be fully generalisable to other departments or polytechnics with different project structures.
- iii. Access to the web application and its live updates depends on internet connectivity and the availability of the managed authentication, database, storage and hosting services.
- iv. The study is an implementation case study rather than a longitudinal investigation of academic outcomes. Its functional results relate to the specified supervision workflow and do not establish general outcomes for every department or academic session.

1.10 Definition of Terms

Digital Project Supervision: The use of a web-based information system, rather than paper records and unscheduled physical meetings, to manage the interaction between a student and an academic supervisor throughout the lifecycle of an HND project.

Progress Tracking: Recording the state of project work and computing its completion percentage from approved chapter milestones; the percentage is distinct from project-health classification and final academic approval.

Milestone: A defined, verifiable project stage with a completion condition and a weight. In this implementation, each of the five chapter stages has a weight of 20 per cent and is complete for progress calculation when its current submission is approved.

Rule-Based Algorithm: An algorithm that applies explicit conditions to stored data. Here, chapter approval conditions produce a progress percentage, and a separate ordered rule set uses activity and defence-related signals to classify project health.

Supervisor: An academic staff user associated with a project in the student's department, responsible for topic decisions, submission review, grading, chapter approval, extension decisions and final approval.

Supervisee: A student linked to a supervisor through a project, either as the lead student or as a registered project partner.

Dashboard: A role-specific screen summarising a user's project records, progress, review workload, notifications or department-level activity.

Notification / Alert: An in-application record of a relevant event, such as partner addition, a topic decision, a submission, feedback or an extension decision. Health badges and reasons indicate schedule or inactivity concerns; email, SMS and push delivery are not part of the implemented notification channel.

CHAPTER TWO

LITERATURE REVIEW

2.0 Introduction

Digital supervision and progress tracking systems for academic projects have attracted growing research attention over the past decade, driven by the recognition that manual, paper-based supervision does not scale well as student enrolment rises and as institutions face increasing pressure to demonstrate the quality and consistency of their project supervision. A parallel body of work has examined rule-based algorithms, expert systems and, more recently, large language model-assisted approaches for automating academic monitoring, feedback generation and progress tracking in educational settings. This chapter reviews the conceptual, theoretical and empirical foundations relevant to a rule-based milestone progress tracking algorithm, critically examines a selection of related published studies, and identifies the specific gap that the present study addresses within the Nigerian polytechnic context.

2.1 Conceptual Framework

The conceptual framework organises the system into Input, System Processing and Output. Inputs include account and department details, student matriculation numbers, the selected supervisor, topic and project-partner records, chapter files, feedback, review decisions, ticket activity, deadlines and a defence date. Processing first verifies the user's role and project access, validates the action, and updates the relevant records. The milestone component counts approved chapter stages and multiplies the count by the fixed 20 per cent chapter weight. Approval of the topic permits chapter work, and approval of all five chapters permits a final submission; the supervisor's final decision is stored separately. A health component evaluates inactivity, submission recency, ticket cadence and defence timing. Outputs are role-specific dashboards, the stage-by-stage breakdown, submission feedback and grades, topic and extension states, activity history, recipient notifications and department summaries. Each new supervision event changes the stored inputs and can therefore change the next displayed output. The framework preserves the supervisor's academic judgement: the rules summarise recorded decisions and activity rather than automatically judging the quality of a report.

2.2 Theoretical Framework

This study is anchored on the Technology Acceptance Model (TAM), proposed by Davis (1989). TAM holds that a user's willingness to adopt and continue using a new information system is shaped primarily by two beliefs: perceived usefulness, which is the degree to which the user believes the system will improve their performance of a task, and perceived ease of use, which is the degree to which the user believes the system requires little effort to operate. Applied to this study, TAM provides the lens through which the eventual adoption of the digital project supervision and progress tracking system by supervisors and students at Yaba College of Technology can be evaluated: a supervisor is more likely to abandon paper logbooks in favour of the platform if it is perceived as genuinely reducing the effort involved in tracking multiple students, and a student is more likely to use the system consistently if the interface for submitting work and viewing progress is perceived as simple and intuitive. Abubakar and Muhammed (2022) applied TAM in a Nigerian tertiary institution to examine the acceptance of internet-based tools for academic purposes and found that perceived usefulness and perceived ease of use remained the dominant predictors of actual system use among Nigerian students and staff, a finding that supports the relevance of TAM to the present study and informs the emphasis placed on a simple, low-friction interface in the design of the proposed system.

2.3 Empirical Framework

A number of empirical studies provide evidence that digital supervision and progress-feedback systems produce measurable improvements over manual processes. van der Merwe, du Toit and Kruger (2020) implemented a model-based academic progress feedback framework at a South African university and, evaluating it through a user survey, found that the framework was readily accepted by staff and students and conformed to the feature requirements identified from prior literature, demonstrating that a structured, computerised feedback mechanism can be successfully embedded into an existing academic supervision workflow. Adu, Onwuegbuzie, Akomolede and Idris-Tajudeen (2024), working within a Nigerian polytechnic context similar to that of the present study, developed a web-based Final Year Project Supervision Management System specifically to address delayed feedback, poor communication and the difficulty of tracking project progress across a large student population, and reported that the resulting system enabled real-time communication and follow-up between students and supervisors regardless of

their physical location. Cavalcanti, Barbosa, Carvalho, Freitas, Tsai, Gašević and Mello (2021), in a systematic review of automatic feedback in online learning environments, found that automated feedback mechanisms consistently reduce the turnaround time between a learner submitting work and receiving a response, while also easing the workload placed on instructors who would otherwise have to provide that feedback manually. Together, these studies support the empirical expectation that replacing manual, meeting-dependent supervision with a rule-based digital platform will measurably improve both the speed of feedback and the visibility of student progress, which is precisely the outcome this study seeks to achieve at Yaba College of Technology.

2.4 Review of Related Work

This section critically reviews twenty published studies, drawn from journals and preprint archives between 2020 and 2026, relevant to digital project supervision, student progress tracking, rule-based algorithms in education, e-learning management systems, and automated academic feedback. Every source cited in this section is listed in full in the References section at the end of this document, and a PDF link is placed beside each study. For each study, the methodology, key findings and a specific limitation or gap relative to the present study are identified.

Adu, Onwuegbuzie, Akomolede and Idris-Tajudeen (2024) developed a web-based Final Year Project Supervision Management System (FYPSMS) for a Nigerian federal polytechnic, using a multi-tier architecture with PHP and MySQL to enable students to upload project drafts, update progress, and receive timely feedback from supervisors regardless of physical distance. Their methodology combined a review of the challenges facing all stakeholders students, supervisors, examiners and administrators with the design and presentation of wireframes for each user role. The system was found to bridge the time and communication gap between supervisors and students, but the study stopped short of implementing any automated, rule-based computation of a numeric progress percentage; progress was represented only as a general status indicator rather than a weighted milestone score, which is precisely the gap the present study's rule-based algorithm addresses.

Castro (2024) designed a Mobile-based Student Internship Monitoring System using a progress tracking algorithm for a Philippine college, allowing trainee students to clock in and out, log daily tasks, and share GPS-verified time records with faculty supervisors,

parents and partner agencies through an Android application. The system was evaluated by thirty stakeholders against the ISO 25010 quality standard and was rated highly for functionality, reliability and usability. However, the progress tracking algorithm in this study was built around attendance and time-logging rather than around discrete academic milestones with defined completion rules, so it is not directly transferable to the chapter-by-chapter, defence-driven structure of an HND research project, which the rule-based milestone algorithm proposed in this study is specifically designed to model.

van der Merwe, du Toit and Kruger (2020) proposed and implemented a model-based framework for computerised academic progress feedback at a South African university, identifying the elements, models and concepts necessary for an effective feedback system from prior literature before implementing and evaluating the framework through a case study and user survey. The framework was readily accepted by end users and conformed to the requirements identified in the literature, offering strong empirical support for the acceptability of computerised progress feedback among academic stakeholders. The framework nevertheless addressed feedback delivery in general terms rather than defining a specific weighted, rule-based scheme for computing a percentage-completion score from individual project milestones, a gap the present study's algorithm fill

Cavalcanti, Barbosa, Carvalho, Freitas, Tsai, Gašević and Mello (2021) conducted a systematic literature review of automatic feedback in online learning environments, synthesising findings from a large body of prior studies on how automated feedback mechanism's function and how effective they are compared with manual instructor feedback. The review found that automatic feedback consistently reduces turnaround time and helps instructors manage large cohorts, but also noted that many of the reviewed systems focus narrowly on quiz or assignment-level feedback rather than on the multi-stage, long-duration structure of a full research project, and rarely incorporate an explicit, auditable rule set of the kind proposed for milestone-based project supervision in this study.

Gorchakov, Demidova and Sovietov (2024) described a rule-based algorithm and its specialisations for measuring the complexity of software written by students in an educational digital environment, expressing established metrics such as Cyclomatic and Cognitive Complexity through the algorithm and proposing a new Educational Complexity metric for classroom use. Their methodology involved formally defining the rule-based

algorithm and validating it against a dataset of student-written programs. The study demonstrates that a rule-based approach can produce transparent, explainable scores in an educational setting, which supports the design choice of this study, but it was applied to source-code complexity rather than to project-milestone progress, and did not address supervisor-student communication or notification at all.

Angeioplastis, Konstantakis and Tsimpiris (2026) audited early-warning models built on Learning Management System log data across three institutional datasets, distinguishing predictive accuracy within familiar courses from reliability when a model is applied to an unseen course. Using several machine-learning model families evaluated under random-row, unseen-student and unseen-course validation protocols, the study found that high accuracy under conventional validation did not guarantee reliable early-warning performance once a course-level distribution shift was introduced. This finding is a caution against relying on opaque predictive models for progress or risk scoring in an academic setting, and reinforces the present study's decision to use a transparent, rule-based algorithm whose milestone weights and thresholds can be audited directly, rather than a statistical model whose behaviour on new cohorts cannot be easily verified in advance.

Zhang, Lee, Xu, Chen and Obaidat (2024) proposed an effective Learning Management System incorporating an advanced educational data mining module capable of extracting frequent and exception rules directly from stored student performance records, rather than relying on an external mining step. Their methodology combined probability-based frequent-pattern mining with iterative updates to the LMS database. The study confirms that embedding rule discovery directly within an LMS improves the efficiency of extracting actionable patterns from student data, but its focus was on mining behavioural and performance patterns from coursework rather than on tracking the discrete, sequential milestones of an extended research project such as an HND dissertation.

Chandra Mouli, Paramesh, Niharika and colleagues (2025) developed a Project Tracker Management Tool intended to track and monitor final-year academic projects, giving guides, coordinators, Heads of Department, principals and directors oversight of a project through its full lifecycle using a full-stack web application modelled loosely on industrial project-tracking tools such as JIRA. The study identifies milestone tracking as a core element that lets both students and supervisors see progress against predetermined objectives and enables early detection of problems. This work is closely aligned with the

present study in intent, but it does not specify a weighted, rule-based algorithm for computing a numeric progress percentage from milestone completion, leaving the precise computation of "how far along" a project is largely undefined — the exact gap this study's algorithm is designed to close.

Chacko, Wang, Perez-Felkner, Haiduc, Whalley and Liu (2025) presented CS-Guide, a framework that combines large language models with weekly self-reported grades and reflective journal entries to trigger tailored academic interventions for computer science students, evaluated on a four-year, approximately twenty-thousand-entry longitudinal dataset with an intervention-recommendation F1 score of up to 97 percent for first-year students. The study found that combining structured, rule-triggered interventions with unstructured qualitative data substantially improved the timeliness and consistency of academic monitoring compared with a human counsellor-to-student ratio that routinely exceeds 350 to 1. The authors also candidly noted that debugging rule-based interventions is time-intensive, since each rule update risks breaking dependencies elsewhere in the rule set a practical concern the present study addresses by keeping the milestone rule set for project supervision deliberately small and independently verifiable.

Kuzminykh, Nawaz, Shenzhang, Ghita, Raphael and Xiao (2024) developed a personalised feedback framework for online education programmes that extends a generative-AI chat model with embeddings to deliver topic-targeted feedback on quiz-based assessments, reporting efficacy rates of 90 percent and 100 percent for open-ended and multiple-choice questions respectively in a proof-of-concept evaluation. Their methodology involved building and testing the framework against a set of assessment items rather than against a full academic-project workflow. The study demonstrates the feasibility of automated, personalised feedback at scale, but is oriented toward assessment marking rather than the ongoing, milestone-based supervision of an extended piece of independent research such as an HND project.

Li, Chen, Tang, Švábenský, Deguchi, Yamashita and Shimada (2025) compared single-agent and multi-agent large language model strategies for automatically assessing open-text student reflections and predicting academic performance, using a dataset of 5,278 reflections from 377 students across three academic terms. The study found that a single-agent, few-shot prompting strategy achieved the closest match to human evaluations, and that models incorporating LLM-assessed reflection scores outperformed baseline models

at identifying at-risk students. While methodologically rigorous, the study concerns the automated scoring of free-text reflections rather than the tracking of discrete, verifiable project milestones, and does not propose any rule-based weighting scheme comparable to the one required for HND project supervision

Qian, Cheng, Guan, Dai, Jin, Yang, Nawaz, Swiecki, Chen, Yan and Gašević (2025) investigated comprehensive, automated evaluation of large language model-generated educational feedback using dedicated LLM feedback evaluators, addressing the practical problem that manually checking the quality of AI-generated feedback at scale is itself labour-intensive. Their methodology involved constructing evaluator models and comparing their judgements against human assessments of feedback quality. The study is valuable for establishing how automatically generated feedback can itself be checked for quality, but it does not address how project milestones should be defined, weighted or aggregated into an overall progress score, leaving the specific rule-based computation this study proposes unaddressed.

Adeyemi and AlOtaibi (2025) designed a feedback-driven decision support system for dynamic student intervention that uses a LightGBM-based regressor with incremental retraining, allowing educators to feed updated post-intervention results back into the model so that its predictions of student performance improve over time, and incorporating SHAP-based explanations for interpretability. The study reported a 10.7 percent reduction in prediction error after retraining. This closed-loop, feedback-driven design is conceptually close to the present study's aim of continuously updated progress computation, but the underlying mechanism is a statistical regression model rather than an explicit, human-auditable rule set, which makes it harder for a project coordinator to explain to a student exactly why their computed status changed a transparency the rule-based algorithm in this study is designed to preserve.

Abiola, Saheed and Akinola (2024) built a Student Project Management System for monitoring undergraduate projects in the Department of Computer Science at the University of Ibadan, using HTML, CSS, Bootstrap, PHP and MySQL to give students, supervisors and coordinators a centralised platform for interacting, collaborating and monitoring project progress. The study identified communication gaps and the absence of a comprehensive monitoring mechanism as major obstacles to timely feedback in the existing manual process at the case-study institution, echoing findings reported elsewhere

in the Nigerian university system. As with several of the other Nigerian case studies reviewed here, the system provides collaboration and document-management features but does not define an explicit rule-based scheme for weighting milestones or computing a numeric progress score, which remains the specific contribution of the present study.

Abubakar and Muhammed (2022) applied the Technology Acceptance Model to examine the factors predicting internet use for academic purposes among students of a Nigerian College of Education, using a survey instrument to measure perceived usefulness and perceived ease of use as predictors of actual system use. The study confirmed that both TAM constructs remained strong, statistically relevant predictors of technology adoption among Nigerian tertiary-level users. This study does not concern project supervision directly, but its findings on Nigerian students' technology-acceptance behaviour directly informed the theoretical framework adopted in Section 2.2 of this study and the emphasis placed on interface simplicity in the proposed system's design.

Hariyanto, Budiman, Yulianto and Yasin (2025) designed a web-based information system for monitoring final projects at STMIK Jayakarta, an Indonesian institution where final-project monitoring was handled manually through Google Classroom, Google Forms and WhatsApp, which made student progress difficult to track and created data-security and communication problems. Their methodology combined observation, interviews and document analysis with Unified Modeling Language (UML) modelling to specify requirements and design the system across six stages of the final-project process, from registration and guidance to defence and post-defence revision. The authors concluded that the proposed design would improve tracking, security and communication. However, the work stops at system design: no application was built or tested, and progress is followed through stage records rather than through a weighted, rule-based percentage computed from milestone completion, which the present study implements and evaluates.

Haris, Abdullah Zailani, Hasim and Abdul Rahman (2025) proposed an integrated platform for managing final year projects at the Malaysian Institute of Information Technology, Universiti Kuala Lumpur, motivated by poor communication, difficulty in monitoring progress and a lack of consolidated documentation and evaluation between students and supervisors. The platform uses web technologies to streamline coordination among supervisors, final year project coordinators and students in supervising, monitoring and evaluating progress, and the authors report that it offers a better process flow for all

stakeholders. The study is closely aligned with the present one in its stakeholder structure, but its reported contribution is a coordination and process-flow platform rather than an explicit, weighted rule set for computing a numeric progress score from milestones, which is the specific contribution of this study.

Isa, Othman, Ali, Azizan and Ferguson (2024) developed a prototype web-based Final Year Project Management System to monitor student performance in the Faculty of Computing and Multimedia at Universiti Poly-Tech Malaysia, where rising student numbers each semester had turned manual project-management procedures into a major problem and made ineffective supervision and monitoring of student progress more likely. The study positions a centralised system as the remedy and demonstrates it through a prototype, which supports the argument that manual monitoring does not scale with enrolment. As a prototype-stage study, however, it reports no weighted milestone scheme or rule-based computation of percentage completion, and its monitoring is not tied to the defence-driven stages of an HND project.

Tavares and Soares (2025) developed and evaluated a large language model-powered tool that gives students formative feedback on their biweekly progress reports in an engineering Capstone programme at Insper in Brazil, where each student's advising professor monitors progress through regular written reports. Using usage data from two rounds of reporting among 76 students, they found an initial barrier to adoption and low engagement, but students who used the tool improved the completeness and quality of their reports, and the tool's task-parsing ability also helped reveal organisational problems in student work. The study supports the value of regular, structured progress reporting, but its tool targets the quality of report writing rather than the measurement of project completion, and its reliance on a generative model makes its outputs harder to audit than the explicit rules proposed in this study.

Shakil and Denny (2024) reported the iterative development of a large computer science capstone course at the University of Auckland over six consecutive sessions as enrolment grew substantially, describing the pedagogies used to sustain student engagement and motivation, as assessed through end-of-course surveys and students' written reflections. Continuous monitoring in the course relied on weekly progress meetings and fortnightly student journals, and the authors note that coordinating a large cohort demands considerable staff time and energy. The study therefore reinforces the

coordination burden that motivates the present work, but as a pedagogical experience report it describes no software system or algorithm for tracking progress, leaving the automation of milestone monitoring unaddressed.

Summary of Reviewed Literature

S/N	Author/Year	Focus	Methodology	Key Finding	Limitation/Gap
1	Adu et al. (2024)	Digital project supervision (Nigeria)	System design; multi-tier web architecture	Bridges time/communication gap between students and supervisors	No rule-based numeric progress score
2	Castro (2024)	Mobile progress tracking (internship)	Mobile app; ISO 25010 evaluation	GPS-verified tracking rated highly usable and reliable	Attendance-based, not milestone/defence-based
3	van der Merwe et al. (2020)	Computerised progress feedback	Framework design; case study; survey	Framework readily accepted by staff and students	No weighted milestone rule scheme
4	Cavalcanti et al. (2021)	Automated feedback (systematic review)	Systematic literature review	Automatic feedback reduces turnaround time, eases workload	Focus on assignment-level, not project-level, feedback
5	Gorchakov et al. (2024)	Rule-based algorithm (code complexity)	Algorithm design; dataset validation	Rule-based scoring gives transparent, explainable metrics	Applied to code complexity, not project milestones
6	Angeioplastis et al. (2026)	LMS early-warning models	Cross-dataset ML audit	High accuracy does not guarantee reliability on unseen courses	Motivates transparent rules over opaque ML models
7	Zhang et al. (2024)	E-learning management system	Embedded rule-mining module	Improves efficiency of extracting behavioural rules from LMS data	Mines coursework patterns, not project milestones
8	Chandra Mouli et al. (2025)	Final-year project tracker	Full-stack web application	Milestone tracking aids early problem detection	No defined weighting/scoring algorithm
9	Chacko et al. (2025)	LLM academic monitoring feedback	LLM + longitudinal dataset (4 yrs)	Up to 97% F1 in intervention recommendation	Rule maintenance is time-intensive at scale

S/N	Author/Year	Focus	Methodology	Key Finding	Limitation/Gap
10	Kuzminykh et al. (2024)	Personalised e-learning feedback	Generative-AI feedback framework; proof of concept	90-100% efficacy on quiz-based feedback	Assessment-focused, not project-supervision-focused
11	Li et al. (2025)	Automated reflection assessment	Single- vs multi-agent LLM comparison	Single-agent, few-shot best matches human raters	No milestone weighting scheme proposed
12	Qian et al. (2025)	Evaluating LLM-generated feedback	LLM feedback evaluators	Enables scalable quality-checking of AI feedback	Does not address milestone definition or weighting
13	Adeyemi & AlOtaibi (2025)	Feedback-driven decision support	LightGBM regressor; incremental retraining; SHAP	10.7% reduction in prediction error after retraining	Statistical model less auditable than explicit rules
14	Abiola et al. (2024)	Undergraduate project monitoring (Nigeria)	Web application (PHP/MySQL/Bootstrap)	Centralises collaboration; improves visibility	No rule-based progress percentage computation
15	Abubakar & Muhammed (2022)	TAM / technology acceptance (Nigeria)	Survey; TAM constructs	Perceived usefulness/ease of use predict adoption	Not project-supervision specific; informs UI design only
16	Hariyanto et al. (2025)	Final-project monitoring system design (Indonesia)	UML-based design; observation and interviews	Web-based design improves tracking, security and communication	Design only; no weighted progress score
17	Haris et al. (2025)	Integrated final year project platform (Malaysia)	Web platform for supervisors, coordinators and students	Streamlines supervision and progress monitoring	No explicit weighted milestone rule scheme
18	Isa et al. (2024)	Final year project management prototype (Malaysia)	Prototype web system development	Manual monitoring does not scale with enrolment	Prototype level; no rule-based progress percentage
19	Tavares & Soares (2025)	LLM feedback on capstone progress reports	LLM tool; usage study (76 students)	Engaged users improved reports; initial adoption low	Targets report writing, not milestone completion

S/N	Author/Year	Focus	Methodology	Key Finding	Limitation/Gap
20	Shakil & Denny (2024)	Large-scale capstone course coordination	Experience report; surveys and reflections	Scaling needs engagement strategies and heavy coordination	No software tracking system or algorithm

2.5 Gap in Literature

The reviewed literature identifies complementary concerns: communication and document management in project-supervision systems; clear, interpretable rules for educational monitoring; and scalable feedback or early-warning approaches. The present study addresses their intersection through a department-scoped supervision workflow in which students select supervisors, propose topics, form registered teams, and submit work for explicit academic review. The implemented progress measure uses five equally weighted chapter approval states rather than a statistical prediction. A separate health measure explains inactivity and defence-related concerns, while notifications and an activity log record the decisions behind the displayed states. This combination provides an institution-specific implementation for the Computer Science HND case study. It is an applied integration contribution; it does not establish that no other system has similar features, or that equal chapter weighting has been empirically shown to be optimal. Institutionally validated weights and a comparative user evaluation remain possible extensions.

CHAPTER THREE

SYSTEM ANALYSIS AND DESIGN

3.0 Introduction

This chapter presents the analysis and design of the Digital Project Supervision and Progress Tracking System for the Department of Computer Science, Yaba College of Technology. It translates the supervision concerns identified in Chapters One and Two into requirements, service relationships, data structures, access controls and explicit progress rules. The design descriptions correspond to the current application source rather than the superseded email-based assignment workflow.

The system supports students, supervisors and the Head of Department (HOD). A student can identify supervisors in the same department, submit a topic, include registered project partners and work in the shared project workspace. The assigned supervisor decides the topic, reviews and grades submissions, approves chapter stages, decides extensions and grants final approval. The HOD monitors projects and supervisory activity in the same department. A separate project-coordinator account type is not implemented; departmental oversight in this version is provided through the HOD role.

The chapter proceeds from methodology and existing-system analysis to the proposed workflow, requirements, architecture, data design, milestone and health rules, security controls and implementation tools.

3.1 System Development Methodology

A sequential system-development lifecycle provided the initial structure for the work. Requirements analysis was followed by system design, implementation, testing and deployment, consistent with a Waterfall organisation of the main deliverables. Subsequent corrections were incorporated through revisions to affected requirements, data handling and interfaces, followed by regression checks. This description refers to the development process; it does not imply that deployment prevents later maintenance or that all requirements were known without revision.

Requirements analysis identified the information and decisions needed by each role. Students require a departmental supervisor directory, a topic-review state, access for their project partners, a chapter breakdown and retrievable feedback. Supervisors require student matriculation numbers, submission review and grades, activity history, deadline control and final-approval conditions. Department administration requires project-health and

supervisory-workload summaries. Authentication, project membership and department scope must be checked because student records and submitted documents are private.

System design specified role-specific routes, trusted server mutations, Firestore project records and subcollections, private file transfer and deterministic progress rules. Transactional creation, submission and review operations maintain related records and counters together. Project health is evaluated on supported operations and through the configured daily refresh. Chapter Four documents the resulting application modules and the completed verification and acceptance exercise.

Verification combined repeatable tests of application handlers, public-page browser checks and functional acceptance of the deployed workflows. The automated tests use controlled service dependencies to exercise validation and state transitions; the browser checks assess layout and signed-out behaviour; and the functional exercise evaluates the interactions between the deployed interfaces and their services. The reported results identify their method so that algorithm checks, interface observations and functional acceptance are interpreted appropriately.

3.2 Analysis of the Existing System

The existing project-supervision process described in Chapter One is largely manual. Students prepare project documents and arrange meetings with supervisors; progress notes and correction history may be kept in personal logbooks, notebooks or separate files. Supervisors coordinate several projects while relying on direct meetings and messages to learn whether students have completed key work. Department administrators may have to request updates individually because project information is not presented in one shared system.

This arrangement creates several operational problems. Records can be incomplete or difficult to retrieve, especially when the person holding the record is unavailable. There is no consistent mechanism for showing the latest project status to all authorised parties. Students may not know whether a submission has been received or when feedback is available. A department-level view of inactivity, unresolved issues and supervisor workload is also difficult to assemble from separate records.

The manual process remains useful for face-to-face supervision, but it does not provide a dependable, timestamped digital record. The proposed system is intended to supplement supervision by centralising project records and making communication events

visible. It does not replace academic judgement or determine whether a student has met the substantive quality requirements of a project.

3.3 Analysis of the Proposed System

The proposed system is a responsive web application with role-specific dashboards. A student first registers a valid matriculation number and signs in. The system returns supervisors from the student's department, and the student selects one before entering a topic and description. Up to four registered partners can be specified by matriculation number. The server checks the accounts and assignments, creates a pending topic and links all accepted members to the same project. The supervisor receives a submission notification and each added partner receives a project-membership notification. Supervisor-initiated project creation remains available as an alternative for an existing departmental student, with the student identified by matriculation number in the interface.

The supervisor can approve a topic, decline it with feedback or leave it pending. A lead student can revise a declined topic and return it for review. An approved topic permits chapter work, and the project workspace presents five chapter states, submission versions, feedback and grades. An approved chapter contributes 20 per cent to progress; submitting a newer version returns that stage to review until the new version is approved. All five stages must be approved before final submission and final approval. Students can request deadline extensions, which the assigned supervisor may grant or decline. Final approval, chapter progress and project health remain separate values, and defence scheduling records a date without implementing a defence panel or result module.

The implementation separates presentation, trusted operations and persistent services. Next.js provides the interface and API routes. Firebase Authentication identifies users and supplies role and department claims; Firestore stores project, team and interaction records; a private Supabase Storage bucket holds document bytes; and Vercel hosts the application and scheduled health refresh. Notifications are recipient records inside the application, while project activity records preserve topic, submission, review and extension events. The HOD workspace aggregates departmental project and supervisor activity.

3.4 System Requirements

3.4.1 Functional Requirements

ID	Requirement
FR1	Allow new users to register a role and department; require a valid, unique matriculation number for students.
FR2	Support student matriculation-number/password login and account email/password login; verify role-specific access and clear client/server sessions on logout.
FR3	Show students the registered supervisors in their own department and allow supervisor selection and topic submission.
FR4	Allow the lead student to include up to four eligible registered partners by matriculation number; link each member to the shared project and notify added partners.
FR5	Allow only the assigned supervisor to approve, decline or mark a topic pending; allow the lead to revise a declined topic.
FR6	Present five chapter states and derive progress at 20 per cent per approved chapter.
FR7	Store project files privately, retain submission metadata and versions, and support chapter revisions through authorised upload and download routes.
FR8	Allow the assigned supervisor to decide submission review status, enter feedback and record a grade from 0 to 100.
FR9	Support submission-scoped comments, recipient notifications, project tickets and timestamped workflow activity.
FR10	Allow project students to request deadline extensions and the assigned supervisor to grant or decline them.
FR11	Require all five chapter stages to be approved before final submission and final approval.
FR12	Allow the supervisor to set the next deadline and defence date, and display those dates to authorised participants.
FR13	Identify students and teams by matriculation number in lecturer project views and roster.
FR14	Provide the HOD with project, supervisor and activity summaries scoped to the same department.
FR15	Evaluate project health from explicit activity and schedule rules during supported operations and the configured daily refresh.

3.4.2 Non-functional Requirements

Quality attribute	Requirement
Security	Verify authenticated sessions, roles, membership and department; use private file storage and server-validated workflow mutations.
Usability	Provide role-specific navigation, legible phone inputs, wrapping layouts and mobile navigation without page-width overflow.
Availability	Use hosted managed services and show actionable errors for unavailable reads, writes or file transfers.
Integrity	Validate matriculation identifiers, partner assignments, grades, dates and chapter/final gates; update related records atomically where implemented.
Auditability	Retain actors, timestamps, review outcomes, submission versions and decision notes in authorised project history.
Maintainability	Separate interface, typed models, server validation, storage access and deterministic progress/health logic.
Performance	Use paginated lists and stored dashboard aggregates to avoid reading every project for each summary view.

3.5 System Architecture

The architecture separates the responsive browser interface, authenticated server logic and external data services. Firebase Authentication supports email/password sign-in; student matriculation-number login is resolved and verified through a server endpoint before a Firebase client login is established. The browser synchronises the authenticated identity with an HttpOnly server session cookie. Protected layouts and API routes verify that session and apply role, department and project-membership checks. Firestore live reads are constrained by database security rules, while workflow mutations use trusted server routes.

Submission files are stored in a private Supabase Storage bucket. The browser obtains authorised signed upload or download access from a server route and transfers the file directly to storage; Firestore retains the file reference and review metadata. This avoids placing file bytes in the main database or making the bucket public. The configured daily Vercel job re-evaluates stored project-health classifications, while relevant writes also invoke the rule engine. Figure 3.1 shows the service relationships and Figure 3.2 shows the principal records. Vercel application deployment and Firebase rule/index deployment are separate operations.

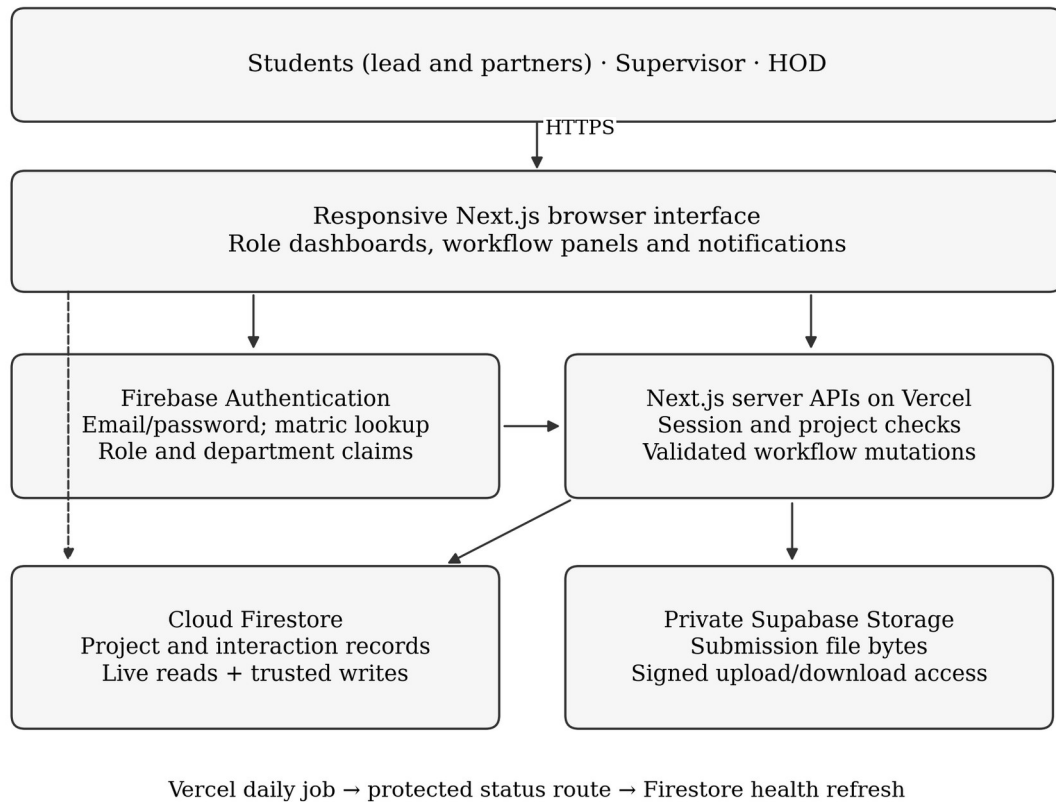


Figure 3.1: Architecture of the Digital Project Supervision and Progress Tracking System

3.6 Database and Data Design

The application uses Firestore documents and subcollections. A user record holds a role, department and, for students, a matriculation number and project link. A server-only matriculation index resolves the student's identifier to a Firebase user ID. Each project identifies its lead student, member IDs, assigned supervisor and department, and stores topic decisions, five chapter states, progress, final approval, extension details, deadlines and activity timestamps. Tickets, submissions and activity entries belong to the project; comments belong to a submission. User-scoped notifications and supervisor/department aggregates support dashboards. Arrays of names and matriculation numbers provide readable team identifiers, while user IDs remain the access-control references.

Collection / record	Main fields or purpose	Relationship
users/{uid}	Name, email, role, department, matricNumber, projectId, supervisorId	User account and assignment; staff views use student matriculation numbers.

Collection / record	Main fields or purpose	Relationship
matric_index/{key}	Normalised matriculation number and uid	Server-only student identifier lookup.
projects/{projectId}	Lead and member IDs; supervisor; topic state; five chapter states; progressPct; finalApproved; defence/deadline and extension fields	Shared project; access follows membership, assignment or department HOD.
projects/{id}/tickets/{ticketId}	Title, body, priority, dueDate, status and author	Project question or task.
projects/{id}/submissions/{submissionId}	Kind, chapterNumber, version, status, grade, reviewFeedback, storagePath and firstResponseAt	Project submission metadata; file bytes reside in private Supabase Storage.
.../submissions/{id}/comments/{commentId}	Author, role, body and timestamps	Discussion attached to one submission.
projects/{id}/activity/{activityId}	Event kind, actor, title, note/date and timestamp	Topic, submission, review and extension audit history.
users/{uid}/notifications/{id}	Kind, title, href, actor, read and createdAt	In-application notification for one recipient.
dashboard_stats/{supervisorId}	Workload, pending reviews, tickets, health counts and response metric	Supervisor aggregate; readable by that supervisor and same-department HOD.
activity_daily/{department_day}	Daily submission, comment and ticket counts	Department-scoped activity chart.

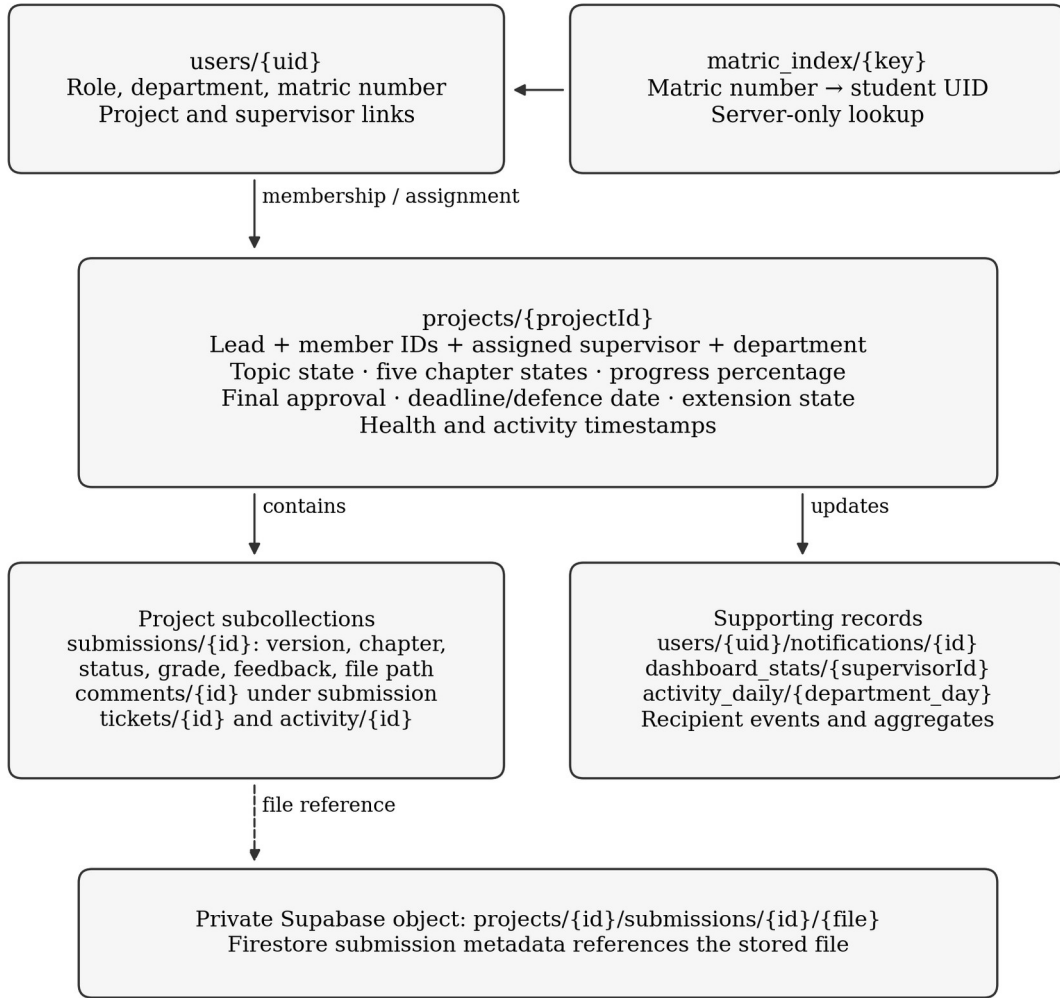


Figure 3.2: Firestore Records, Project Membership and Private Submission Storage

3.7 Rule-based Milestone Progress and Project Health Algorithms

The implementation maintains two related but distinct outputs: chapter-based progressPct and the categorical milestoneStatus used for project health. Let A_i equal 1 when the current state of chapter i is approved and 0 otherwise. For five chapters with equal weights $w_i = 20$, progress is $P = \sum(w_i \times A_i)$, for $i = 1, \dots, 5$. Consequently, zero, one, two, three, four and five approved chapters yield 0, 20, 40, 60, 80 and 100 per cent respectively. Pending review and requested changes contribute no weight. The server derives this value from chapter states; students cannot submit an arbitrary progress percentage.

Topic approval is a prerequisite for chapter work rather than a separately weighted milestone. A chapter or revision submission moves its stage to In review; the assigned supervisor's decision changes it to Approved or Changes requested. The latest submission reference prevents an older chapter review from overwriting the stage of a newer version. Replacing an approved chapter with a new version returns that chapter to review and can

reduce the progress percentage. All five stages being approved permits a final submission, but the supervisor must still approve that final document before `finalApproved` becomes true and the project lifecycle status becomes approved. A displayed 100 per cent therefore indicates approved chapter coverage and does not, by itself, prove final clearance or successful defence.

Project health is evaluated separately using ordered rules. No recorded activity, or inactivity greater than 21 days, yields Stalled. Otherwise, a defence date that has passed without final approval, or a date within the next 30 days without final approval, yields Behind. A further Behind condition applies when defence is within 45 days and no submission exists or the last submission is more than 45 days old. Finally, fewer than one ticket in the preceding 30 days combined with inactivity greater than 10 days yields Behind. A project that matches none of these conditions is On Track. The first matching rule wins, so inactivity takes priority over defence pressure.

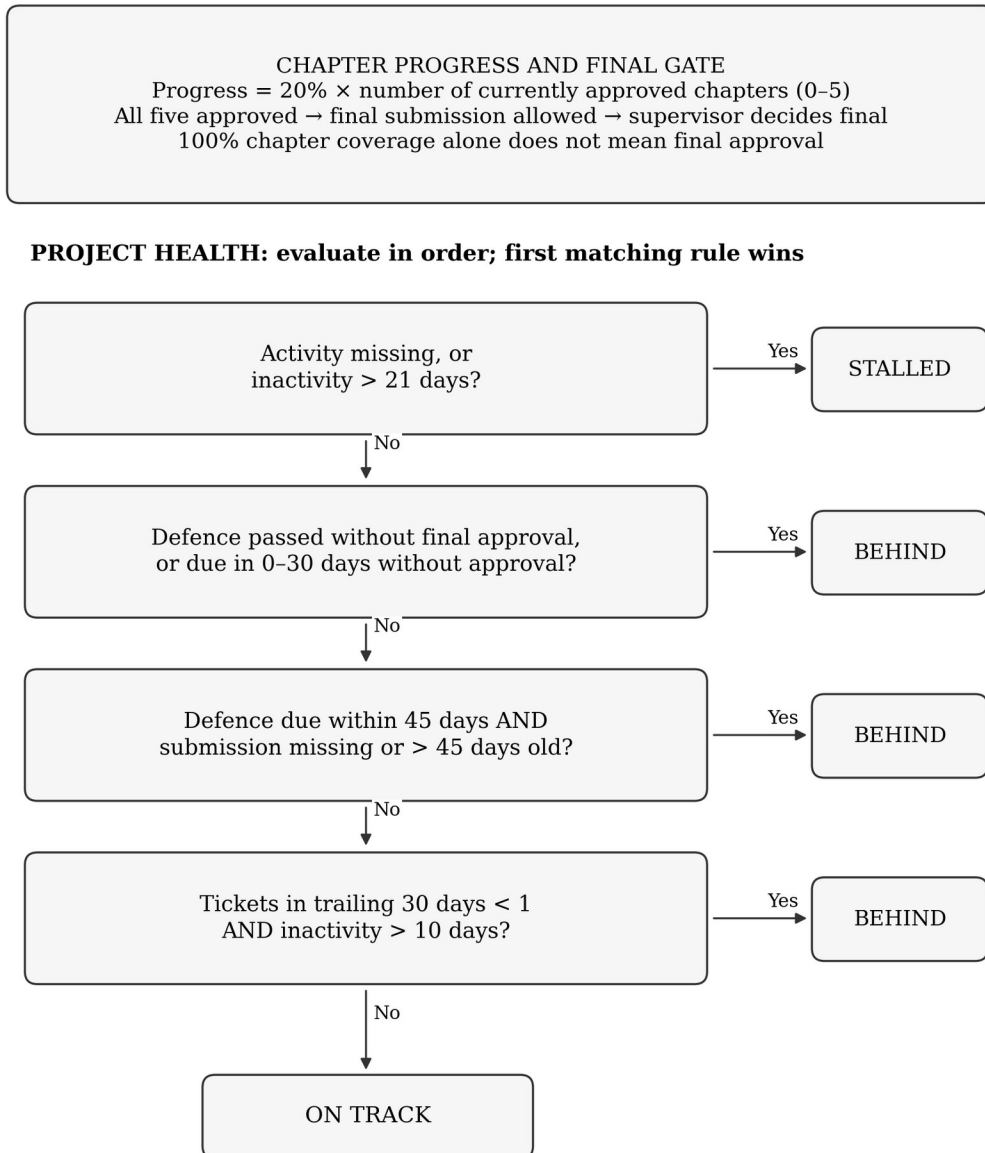


Figure 3.3: Chapter Progress, Final-Approval Gate and Ordered Project-Health Rules

Figure 3.3 summarises both computations. Health indicators are monitoring signals rather than grades: a project can have substantial approved-chapter progress and still be flagged for inactivity. Final approval does not bypass the first inactivity rule. The threshold values and chapter weights are explicit, allowing a reader to trace a displayed result to its stored inputs. Workflow actions create in-application notifications, while the health result and explanatory reason appear in project views and summaries. The daily refresh maintains health classifications without implementing email or SMS reminders.

3.7.1 Progress and Status Evaluation Pseudocode

Algorithm 3.1: Chapter Progress and Project Health Classification

```
FUNCTION computeProgress(chapterStates)
  approvedCount = COUNT(state = APPROVED)
  RETURN approvedCount × 20
END FUNCTION

FUNCTION classifyHealth(project, now)
  IF activityMissing OR inactivityDays > 21 THEN RETURN STALLED
  IF defenseDate exists THEN
    d = daysUntilEndOfDefenseDate(now)
    IF d < 0 AND NOT finalApproved THEN RETURN BEHIND
    IF 0 ≤ d ≤ 30 AND NOT finalApproved THEN RETURN BEHIND
    IF d ≤ 45 AND (submissionMissing OR submissionAge > 45) THEN
      RETURN BEHIND
    END IF
  IF ticketsLast30Days < 1 AND inactivityDays > 10 THEN RETURN BEHIND
  RETURN ON_TRACK
END FUNCTION

FINAL SUBMISSION AND APPROVAL REQUIRE five chapter states =
APPROVED
```

3.8 Security and Access Control

Identity, role assignment and operation authorisation are separate concerns. Firebase Authentication verifies the account, and the provisioning endpoint sets the selected role and department once for a new account. Existing provisioned accounts cannot use that endpoint to change role; protected layouts, server APIs and Firestore rules enforce the resulting permissions. Students access projects through lead or partner membership, supervisors through assignment, and HOD users through department scope. Sensitive project and submission mutations are denied to direct client database writes and are validated by server handlers.

Staff self-registration is available through distinct registered email accounts. Role enforcement prevents a provisioned student account from exercising supervisor actions, and the recorded role cannot be reassigned through the provisioning endpoint. The current registration policy does not use hardcoded staff-email lists or independently verify employment. Submission storage remains private, with temporary signed access granted after a project-authorisation check. Logout clears Firebase client authentication and the HttpOnly server session cookie in a controlled order, then loads a fresh landing page so cached dashboard controls do not remain visible.

3.9 Development and Deployment Tools

Layer	Technology observed in the deployed implementation	Role
Application framework	Next.js 14 App Router	Web pages, routing and server API endpoints
Language	TypeScript	Typed application and domain logic
Interface	Tailwind CSS and shadcn/ui components	Responsive layouts and common controls
Authentication and database	Firebase Authentication and Cloud Firestore	User identity, role claims and structured application data
File storage	Supabase Storage private bucket	Project-submission file storage and signed access
Hosting and scheduled work	Vercel with scheduled cron route	Application deployment and daily status refresh

The implemented stack consists of Next.js 14, React, TypeScript, Tailwind CSS, shadcn/ui-style components, Firebase Authentication, Cloud Firestore, private Supabase Storage and Vercel hosting. Recharts supports dashboard charts, and Node's test runner exercises workflow and session logic. Playwright with Chromium and WebKit provides browser checks. The interface uses locally bundled Roboto and YABATECH green (#006600) and yellow (#FDC800), whereas the project report retains the corrected document's Times New Roman formatting. Daily cron processing is configured in Vercel; the repository also contains Firebase function code, which is not evidence that those functions have been deployed.

CHAPTER FOUR

SYSTEM IMPLEMENTATION, TESTING AND RESULTS

4.0 Introduction

This chapter presents the implementation and testing of the Digital Project Supervision and Progress Tracking System for the Computer Science case study at Yaba College of Technology. It explains how the design in Chapter Three was translated into a working web application and examines the resulting interfaces, workflow controls and monitoring outputs. The discussion follows the project lifecycle from account creation and topic submission to chapter review, extension decisions and final approval.

Verification combined automated regression tests, browser checks and functional acceptance of the deployed application. The project owner completed the functional exercise and confirmed that the requested workflows operated successfully. The supplied screenshots illustrate representative interface states from that exercise. They are discussed according to what each screen displays, while the test-result tables summarise the broader completed checks.

4.1 Implementation Overview

The application uses Next.js 14 with the App Router to serve role-specific pages and authenticated API routes. React and Tailwind CSS implement the interface, while TypeScript defines the principal domain models and validation boundaries. Firebase Authentication supplies account identity and role claims; Cloud Firestore stores users, projects and supervision interactions; and a private Supabase Storage bucket holds submission files. Vercel hosts the application and the configured daily project-health refresh.

The three user roles are Student, Supervisor and Head of Department. The student workspace supports supervisor selection, topic submission, project partners, chapter work, feedback, tickets and extension requests. The supervisor workspace supports topic decisions, submission review and grading, chapter and final approval, deadline control and supervision monitoring. The HOD workspace provides department-scoped project and supervisor oversight. Each role operates on the same underlying project records within its authorised scope.

The implementation distinguishes identity from academic decisions. A matriculation number identifies a student, the project membership determines student access, and the

assigned supervisor determines topic and review outcomes. Chapter progress summarises approved stages, while project health summarises activity and schedule conditions. These distinctions prevent a login role, progress percentage or health badge from being treated as an academic grade or a final-clearance decision.

4.2 User Interface Implementation

4.2.1 Account Registration and Student Identification

The registration interface records the user's name, email, password, department and selected role. A student must also provide a matriculation number. Figure 4.1 shows the role choices and student-registration fields, together with a completed example identifying Computer Science as the department. Staff accounts use registered email addresses without the student matriculation field.

To exit full screen, press and hold **Esc**

Create account

Set up your supervision workspace.

I am a

☒ **Student**
 Submit work, track milestones, raise tickets

☐ **Supervisor**
 Review submissions, run your student roster

☐ **Head of Department**
 Department-wide oversight and analytics

Full name

Ada Okafor

Matriculation number

F/HD/24/3211001

Format: F, D or P / ND or HD / year / 7-digit number.

Email

you@yabatech.edu.ng

Password

At least 6 characters

Department

Select your department

(a) Role selection and student registration fields.

Full name

shazily

Matriculation number

F/HD/24/3999001

Format: F, D or P / ND or HD / year / 7-digit number.

Email

testtest@gmail.com

Password

.....

Department

Computer Science

Create account

Already have an account? [Sign in](#)

(b) Student details, department selection and account creation.

Figure 4.1: Account Registration and Student Identification

The accepted student identifier has the form F/HD/24/3211001. F, D and P denote full-time, distance-learning and part-time programmes; HD and ND identify the diploma category; the third segment is a two-digit year; and the final segment contains seven digits. Normalisation removes incidental spaces and converts letters to uppercase. A server-side index links the identifier to the Firebase user ID and prevents duplicate student matriculation registration.

Students can sign in with a matriculation number and password or with their account email and password. Supervisor and HOD accounts use email and password. Once an account is provisioned, its role cannot be changed through another provisioning request. The registration policy permits new staff accounts to select their role; it does not represent an independent employment-verification process.

4.2.2 Supervisor Selection, Topic Submission and Project Partners

The student dashboard lists registered supervisors from the student's department. Figure 4.2(a) illustrates the directory, while Figure 4.2(b) shows a selected supervisor, a proposed project topic and the partner-entry controls. The demonstration topic concerns a medical-diagnostics and appointment-booking system; it is a sample project record used to exercise the supervision workflow, rather than the subject of this report.

YABATECH PROJECT SUPERVISION

Dashboard

My project

Dashboard
Your project, milestone health and recent activity.

Start your project
Select a supervisor in your department, propose your topic and add registered project partners.

Supervisor

Choose your supervisor

Choose your supervisor

Joy

Mr. Tunde Bakare

Mrs. Ngozi Eze

Omowumi

Shazil Test

Shazily Sanni (Lecturer)

test

Project partners (up to four)

Add project partner

Submit topic for review

YABATECH HND - Computer Science

(a) Supervisors listed for the student's department.

YABATECH PROJECT SUPERVISION

Dashboard

My project

To exit full screen, press and hold **Esc**

Start your project
Select a supervisor in your department, propose your topic and add registered project partners.

Supervisor

Shazily Sanni (Lecturer)

Project topic

AI-based medical diagnostics, and doctor appointment booking systems.

Brief description (optional)

AI-based medical diagnostics, and doctor appointment booking systems.

Project partners (up to four)

F/HD/24/3211001 Remove

F/HD/24/3211001 Remove

F/HD/24/3211001 Remove

F/HD/24/3211001 Remove

Add project partner

Submit topic for review

YABATECH HND - Computer Science

(b) Selected supervisor, proposed topic and partner-entry controls.

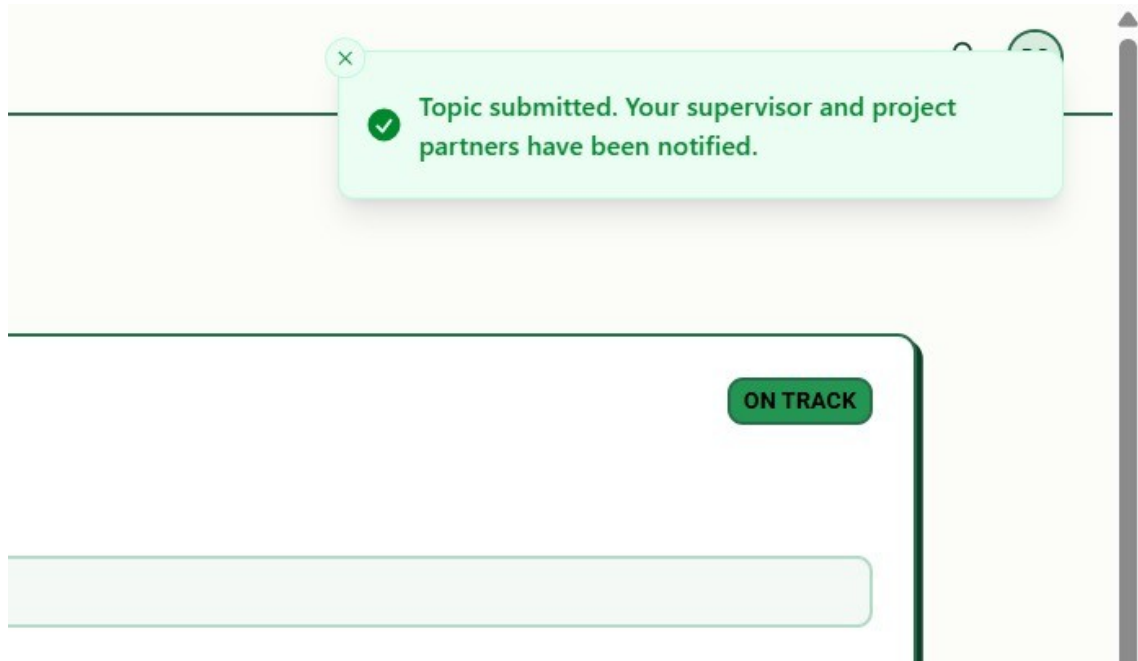
Figure 4.2: Departmental Supervisor Selection and Topic/Team Setup

The lead student submits a topic and optional description after selecting the supervisor. Up to four registered partners can be specified by matriculation number. The server checks the supervisor's role and department, the partner accounts and their current assignments before linking the accepted members to the project. The new topic begins in Pending state, the supervisor receives a notification, and each added partner receives a notification identifying the student who added them.

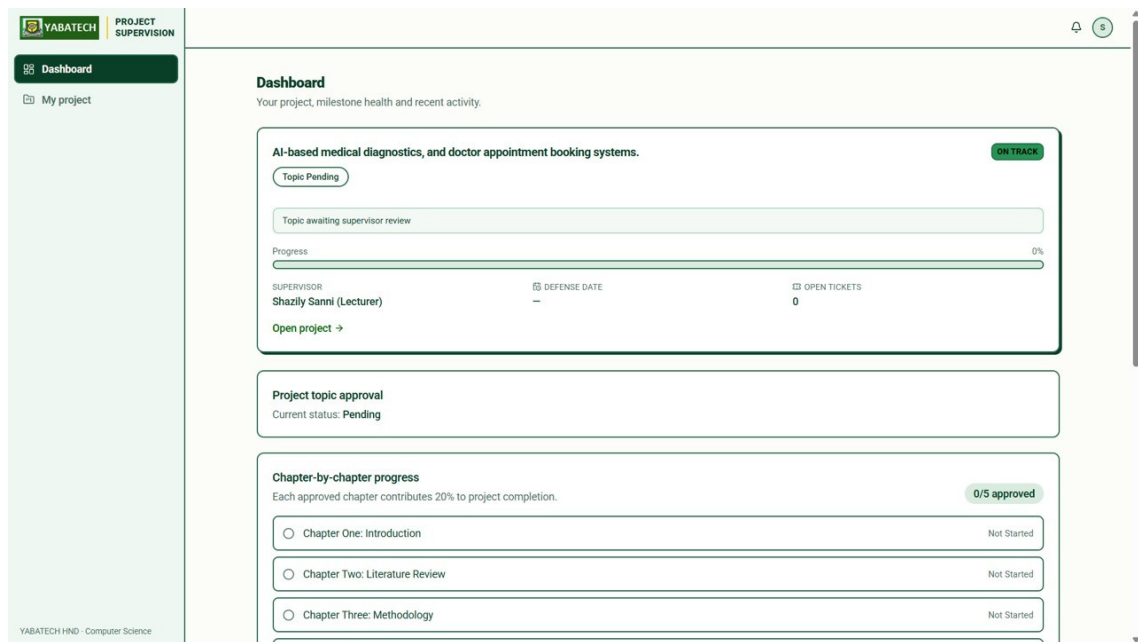
A partner is a member of the shared project, with access to its authorised workspace and file operations. The association is stored on the user and project records rather than relying on a notification link alone. The interface retains names and matriculation numbers for readable team identification, while user IDs determine access.

4.2.3 Submission Confirmation and the Student Dashboard

Figure 4.3(a) displays the successful topic-submission message, which states that the supervisor and project partners have been notified. Figure 4.3(b) shows the resulting student project summary: the topic is Pending, progress is initially 0 per cent, the supervisor is identified, and the health badge is On Track. These values describe the initial state of that example. A pending topic is compatible with an On Track badge because topic approval and project-health classification represent different conditions.



(a) Successful topic-submission confirmation.



(b) Pending topic, initial progress and project summary.

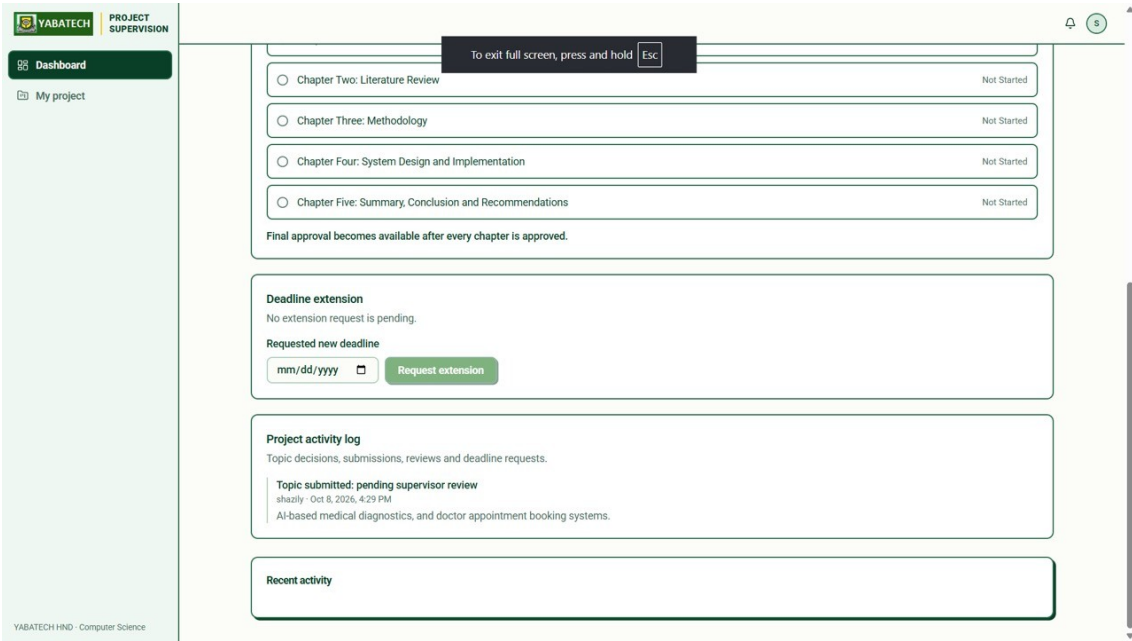
Figure 4.3: Topic-Submission Confirmation and the Student Project Dashboard

The project dashboard presents the current topic decision, the progress bar, supervisor information, defence date when recorded, and open-ticket count. It links to the shared project workspace, where the student can follow chapter states and supervision decisions. The notification feed and activity log provide related but distinct views:

notifications draw a recipient’s attention to an event, while the activity log preserves its project context, actor and timestamp.

4.2.4 Chapter Breakdown, Activity History and Extensions

Figure 4.4 presents the stage-by-stage chapter breakdown, deadline-extension interface and project activity history. The five stored stages are Introduction; Literature Review; Methodology; System Design and Implementation; and Summary, Conclusion and Recommendations. The report retains the corrected academic chapter titles; chapter numbers map submitted documents to these application stages even where the interface label is phrased differently.



Chapter states, deadline-extension controls and timestamped project activity.

Figure 4.4: Chapter Stages, Deadline Extension and Project Activity History

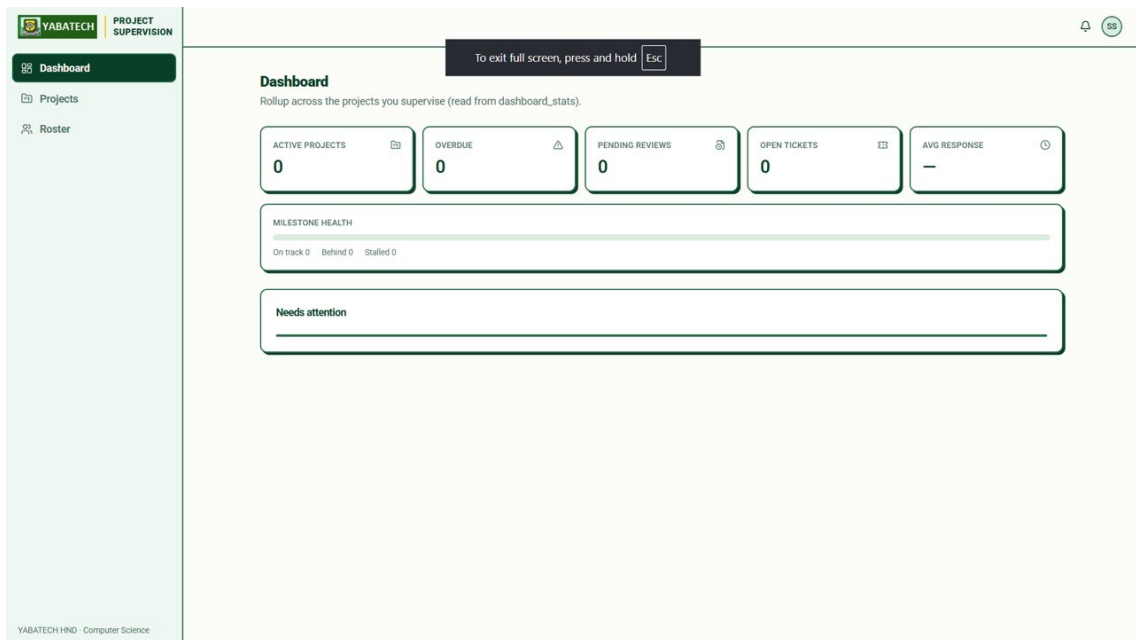
A chapter is Not started before submission, In review after submission, Changes requested when corrections are required, and Approved after the supervisor approves its current version. Each approved stage contributes 20 per cent to project progress. The screenshots show a newly submitted topic with chapter work not yet started and an activity entry for topic submission. They illustrate the initial workflow layout rather than a completed-project result.

The extension form lets a project student propose a later deadline. Validation checks a real date later than the current deadline and within the implemented request horizon; an existing Pending request cannot be replaced by another request. The assigned supervisor

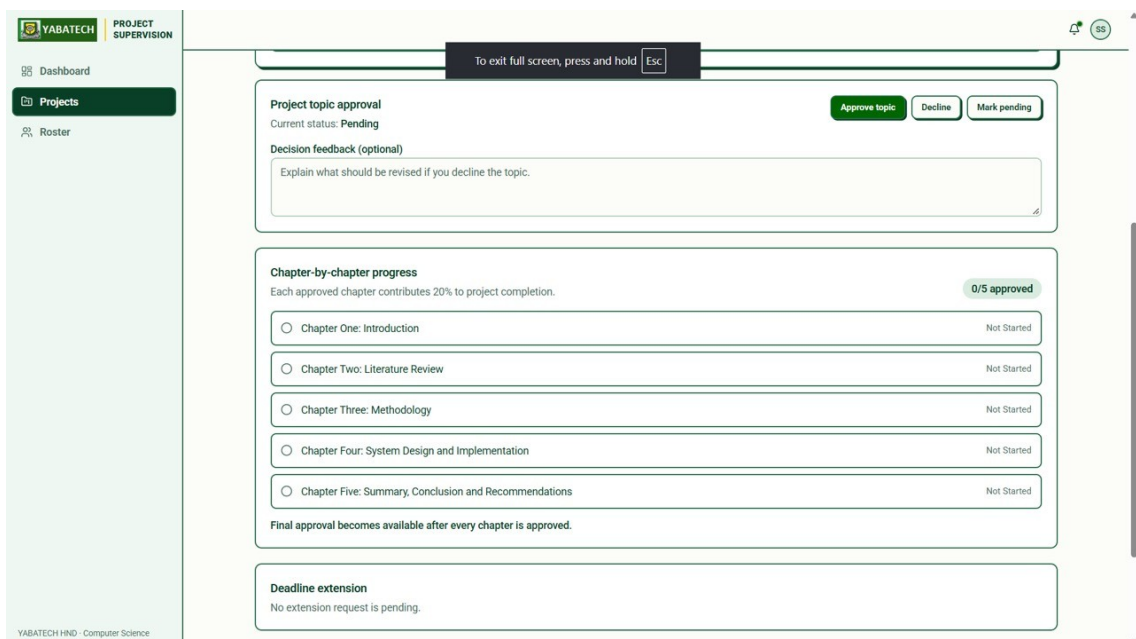
grants or declines the request with an optional note. Approval changes nextDeadline, while decline preserves it. The defence date is maintained separately. Decision notifications and activity records make the outcome visible to the project members.

4.2.5 Supervisor Dashboard and Topic Decisions

Figure 4.5(a) shows the supervisor dashboard before project activity, with zero workload counters and no recorded average response. The dashboard summarises active projects, overdue or warning items, pending reviews, open tickets and project-health counts. Figure 4.5(b) shows the topic-decision workspace: the assigned supervisor can Approve topic, Decline or Mark pending, and can enter decision feedback.



(a) Supervisor workload summary before project activity.



(b) Pending topic with approval, decline and pending controls.

Figure 4.5: Supervisor Dashboard and Topic-Decision Controls

A declined topic may be revised and resubmitted by the lead student, returning it to Pending for review. Chapter work remains blocked until topic approval, although proposal files can be submitted during topic consideration. The decision, note, actor and timestamp are retained in the project record and activity history. Project members receive the relevant decision notification.

4.2.6 Project Review, Feedback, Grades and Defence Scheduling

Figure 4.6 shows the supervisor project overview and its Overview, Tickets, Submissions and Feedback navigation. The captured record identifies a student by matriculation number and displays the team association, progress, open-ticket count and defence-date entry control. Matriculation identity is used in lecturer-facing project and roster views instead of presenting a student email as the principal identifier.

The screenshot displays the 'PROJECT SUPERVISION' interface for YABATECH. The left sidebar contains navigation links: Dashboard, Projects (selected), and Roster. The main content area shows the project overview for 'AI-based medical diagnostics, and doctor appointment booking systems', which is marked as 'ON TRACK'. Below the project title, there are tabs for Overview, Tickets, Submissions, and Feedback. The 'Overview' tab is active, showing a progress bar at 0%, the student's name 'daniel susu' with matriculation number 'P/HD/24/3999003', and the project team 'daniel susu · P/HD/24/3999003'. There is a 'SET DEFENSE DATE' section with a date input field and a 'Save' button. At the bottom, there is a 'Project topic approval' section with a 'Current status: Pending' and buttons for 'Approve topic', 'Decline', and 'Mark pending'. A text area for 'Decision feedback (optional)' is also present.

Project overview showing matriculation identity, team, navigation tabs and defence-date entry.

Figure 4.6: Supervisor Project Overview, Student Identity and Defence-Date Control

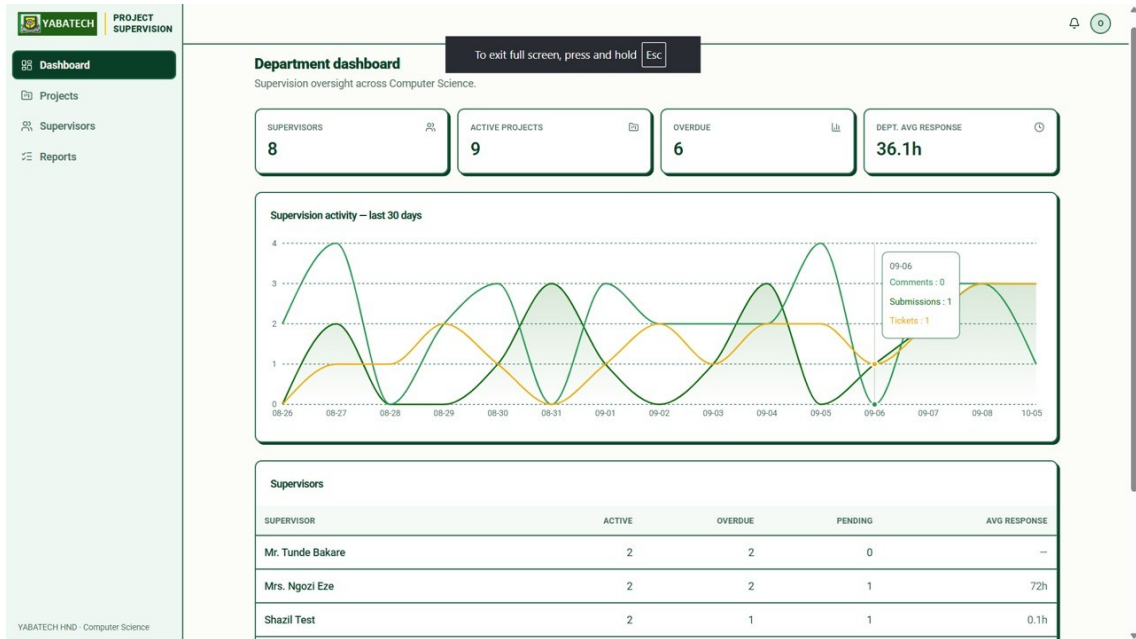
The submission workspace accepts PDF, Word, PNG and JPEG documents up to 25 MB. The browser first obtains authorised signed upload access, transfers the file to the private storage bucket and registers its metadata. A record retains the title, kind, chapter number where applicable, version, submitting user, file reference, review state and timestamps. Version records preserve earlier documents, while the latest-chapter reference identifies the version controlling the current stage.

The assigned supervisor saves a review outcome, written feedback and an optional grade from 0 to 100. Submission-scoped comments support further discussion between authorised participants. A revision creates a new submission and returns its selected stage to review; an older review cannot overwrite a newer chapter stage. Review decisions update the activity history, notify the project members and maintain the associated counters.

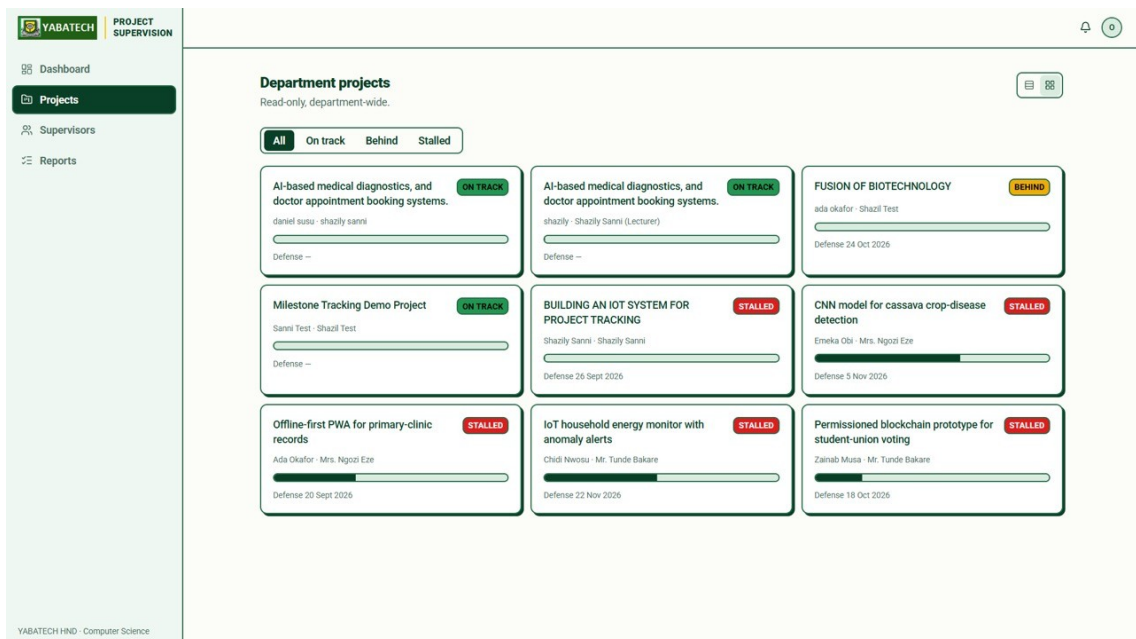
The supervisor can set the next deadline and the defence date as distinct project fields. All five chapters must be approved before a final submission is accepted, and the same condition is enforced for final approval. The supervisor's approval of the final document records the project's explicit clearance state. The system supports defence-date tracking; examiner allocation, panel management, defence results and certification remain outside its defined scope.

4.2.7 HOD Dashboard and Departmental Project Overview

Figure 4.7(a) illustrates the HOD dashboard for Computer Science. In the captured state it displays eight supervisors, nine active projects, six overdue items and a department average response of 36.1 hours. These are the values displayed for that demonstration dataset at capture time, not a measured before-and-after improvement in departmental supervision. The activity chart presents submission, comment and ticket series, while the supervisor table supports follow-up at staff level.



(a) Departmental counters, response metric and supervision-activity chart.



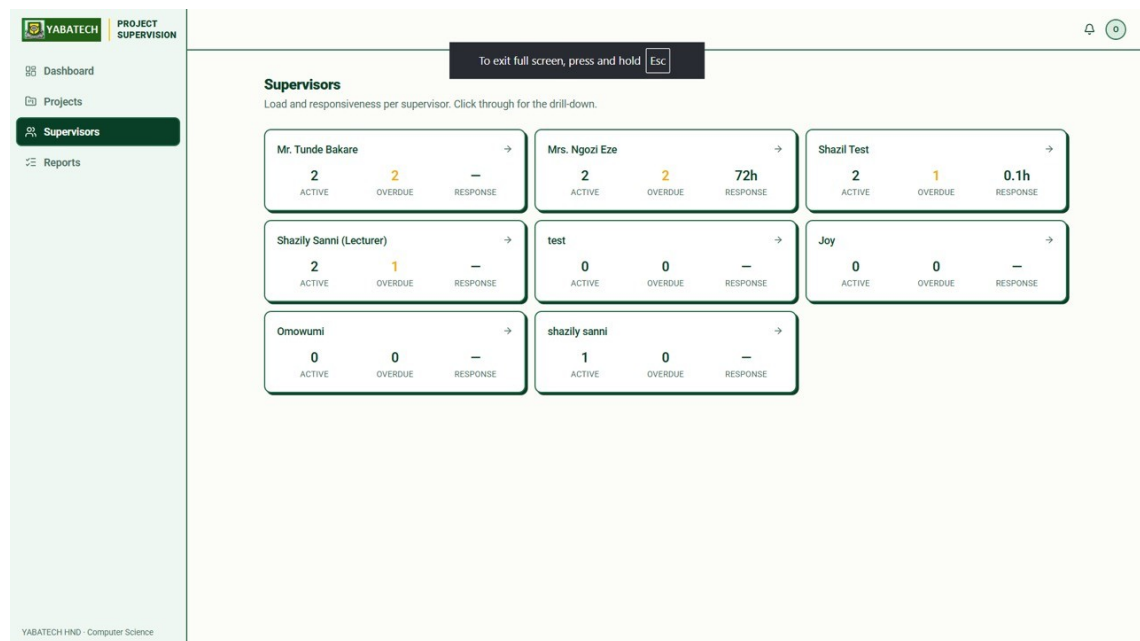
(b) Department projects with progress and health classifications.

Figure 4.7: HOD Activity Dashboard and Departmental Project-Health Overview

Figure 4.7(b) shows the department project view with On Track, Behind and Stalled classifications, progress bars and defence dates where present. The health labels allow the HOD to identify projects requiring attention without replacing academic judgement. The view is department-wide and read-only; the assigned supervisor remains responsible for topic, grading, extension and final-approval decisions.

4.2.8 Supervisor Workload and Responsiveness

Figure 4.8 presents the department's supervisor workload view. Each card summarises active projects, overdue items and recorded response time, and links to further detail. The figure includes both supervisors with active work and accounts with no assigned workload. A dash represents the absence of a recorded response value in that view. Workload and response indicators support departmental follow-up but are not student grades or measures of staff performance independent of the underlying data.



Supervisor workload, overdue-project and response summaries.

Figure 4.8: Departmental Supervisor Workload and Responsiveness

4.2.9 Mobile Presentation and Logout

The interface uses locally bundled Roboto and YABATECH green and yellow. Responsive layouts allow branding, navigation and controls to wrap at phone widths; forms retain legible input text and long content remains within its layout containers. The narrow landing header places account actions separately from the branding so that the dashboard button does not overlap the project title. The academic report retains the corrected document's Times New Roman formatting.

Logout clears Firebase client authentication and the HttpOnly server session cookie, then performs fresh navigation to the public landing page. Session synchronisation prevents a pending or delayed authentication callback from restoring a logged-out session. Initial

signed-out state also clears cookies left by the earlier logout flow, while restored browser-cache pages are reloaded. Protected dashboard access subsequently requires a valid login.

4.3 Testing and Results

The verification strategy combined three methods. Automated regression tests exercised the actual application handlers and session-synchronisation logic with controlled dependencies. Browser checks examined rendered public pages at 320, 375, 390, 430, 768 and 1440 pixel widths. Functional acceptance was performed on the deployed role-based workflows by the project owner, who confirmed that all requested flows operated successfully. This combination assessed validation logic, presentation and operational behaviour.

The regression record contains 34 passing tests: 25 workflow and validation checks and nine session-synchronisation checks, with no failures or skipped tests. Eighteen public-page checks passed against the live deployment in Chromium and 18 passed against a local production build in WebKit over HTTPS. Build, lint and TypeScript checks completed successfully, and the Vercel deployment succeeded. Table 4.1 summarises these results and the completed functional acceptance exercise.

Table 4.1: Verification Evidence and Its Scope

Verification method	Result	Evidence and interpretation
Workflow and validation regression tests	25 passed	Actual application handlers tested with controlled service dependencies.
Session-synchronisation regression tests	9 passed	Logout ordering, delayed callbacks, stale-cookie recovery and session errors.
Live Chromium browser checks	18 passed	Public pages at six viewport widths, layout, cookie deletion and signed-out routing.
Local WebKit browser checks over HTTPS	18 passed	Safari-engine rendering and session-cookie behaviour on the production build.
Build, lint, TypeScript and deployment	Successful	Compilation and static checks completed; Vercel deployment succeeded.
Deployed functional acceptance	Passed	Project-owner completion and confirmation of all requested role-based workflows; supplied screenshots illustrate representative states.

Table 4.2 summarises the functional acceptance outcomes supplied by the project owner. The application figures illustrate selected states from the exercise; the successful acceptance result is reported from the completed functional checks rather than inferred from every screenshot. Separating displayed examples from test outcomes permits a Pending or Not started example to explain the interface without presenting it as the final state of the exercise.

Table 4.2: Completed Functional Acceptance Results

Area	Acceptance criterion	Result
Registration and login	Student matriculation identity and account login operate correctly; staff use their registered email accounts.	Passed
Supervisor/ topic setup	The student selects a departmental supervisor, submits a topic and includes eligible registered partners.	Passed
Membership and notifications	Added partners receive the project event and can access the shared project; notifications identify relevant workflow actions.	Passed
Topic decisions	The assigned supervisor can approve, decline or retain Pending; a declined topic can be revised and resubmitted by the lead.	Passed
Chapter work and review	Document upload, revisions, feedback, grades and current-version review states support chapter progression.	Passed
Progress and final approval	Approved stages produce 20-per-cent increments; all five approved chapters are required before final submission and approval.	Passed
Tickets and activity	Project communication and timestamped decisions are recorded and retrievable by authorised users.	Passed
Deadline and defence date	Extension requests and supervisor decisions work; deadline and defence-date values remain distinct.	Passed

Area	Acceptance criterion	Result
Department monitoring	HOD project, supervisor, workload and activity views provide the authorised department overview.	Passed
Responsive use and logout	Layouts support mobile use and logout returns the account to signed-out behaviour with protected access requiring login.	Passed

4.3.1 Relationship between Results and Study Objectives

The results support the implementation objectives through observable behaviour. Account and department handling establish the supervision roles; supervisor selection and project membership establish the student–supervisor relationship; topic decisions and chapter reviews provide structured academic control; and equal chapter weights produce a reproducible progress percentage. Feedback, grades, notifications, tickets and activity history maintain the communication record, while HOD views provide department-level visibility.

The practical contribution is a working mechanism for recording decisions and displaying progress consistently. The acceptance result supports the suitability of the implemented workflow for its defined tasks. The response-time metric provides information for monitoring supervision; it should be interpreted as a stored operational value rather than a controlled comparison with the previous manual process. The discussion therefore connects the verified functions to the study objectives without inventing a statistical reduction in feedback delay.

4.4 Implementation and System Behaviour

4.4.1 Progress Computation and Approval Conditions

The chapter-progress rule is $P = 20 \times A$, where A is the number of currently approved chapter stages. One approved chapter yields 20 per cent, two yield 40, three yield 60, four yield 80 and five yield 100. A stage In review or requiring changes contributes zero. The equal weights and explicit approval condition make the result reproducible from the stored chapter states, rather than dependent on the number of uploaded files or a manually entered percentage.

For example, Chapters One and Two approved with Chapter Three In review produce 40 per cent progress. A new submission replacing approved Chapter Two returns

that stage to review and temporarily reduces the count to one approved stage. When all five chapters are approved, progress is 100 per cent; final approval remains a separate supervisor decision on the final document. This separation distinguishes chapter coverage from project clearance and the scheduled defence.

4.4.2 Project-Health Classification

The health engine applies ordered conditions to activity and schedule data. Missing activity or inactivity exceeding 21 days yields Stalled. Otherwise an unapproved final after the defence date, or within the next 30 days, yields Behind. Defence within 45 days combined with a missing or more-than-45-day-old submission provides another Behind condition. Fewer than one ticket in the preceding 30 days combined with inactivity exceeding 10 days also yields Behind. The default result is On Track.

The first matching condition wins, and the result includes an explanatory reason. A scheduled refresh keeps the health state current as calendar conditions change, while supported operations also invoke the engine. Approval and health fields remain separate: a finally approved project can later be flagged for inactivity under the first rule. The HOD should therefore read the health signal alongside lifecycle status, chapter coverage and final approval.

4.4.3 Data Integrity and Private File Handling

Trusted server routes validate workflow actions and update the associated records. The student creation transaction links the project, member assignments, activity and notifications. Submission and review transactions maintain the current stage, progress, review outcome and counters. The final gate checks all five chapter states, while the latest-chapter reference prevents an older submission decision from replacing the current stage. Invalid grades, inappropriate assignments and unauthorised academic decisions are rejected.

Firestore retains structured records and file references, while the private Supabase bucket retains the document bytes. Server signing routes verify project access before granting temporary upload or download access. Members, the assigned supervisor and the same-department HOD have defined read scope, and sensitive project and submission mutations use server APIs. Firestore rules and indexes are managed for the matching database separately from the Vercel application deployment; submission storage remains private.

In-application notifications are recipient records and can be marked read by their owner. Project activity entries preserve the actor, timestamp and decision context. Stored response timestamps and daily activity aggregates support supervisory monitoring. Together, these mechanisms provide traceable records for topic decisions, submissions, feedback, extensions and approvals without making the document files publicly accessible.

4.4.4 Discussion

The implementation addresses the fragmented-record problem by keeping project identity, membership, decisions and documents in a coordinated workspace. Students can identify the current academic state instead of relying only on an informal message; supervisors can retrieve the current submission and its history; and the HOD can inspect department-wide activity and warning states. The rule-based progress calculation makes approval coverage visible, while health classifications direct attention to inactivity and defence pressure.

The completed acceptance exercise confirms that the requested functional workflow operates on the deployed application. Automated checks provide repeatable coverage of validation and important failure paths, and the browser checks support responsive public-page presentation and signed-out behaviour. These results demonstrate the technical contribution of the system within the study scope. They do not turn dashboard values into evidence of a statistically controlled institutional performance experiment.

4.5 User Guide

4.5.1 Student

- i. Register with a name, account email, password, department and valid matriculation number. Use the matriculation number and password to sign in, or use the account email option.
- ii. If no project exists, review the supervisors in the department, select one, enter a topic and description, and optionally add eligible registered partners by matriculation number. Submit and confirm the topic is Pending.
- iii. Read the supervisor's topic decision and note. The lead can revise a declined topic and resubmit it. Wait for approval before submitting chapter work.

- iv. Open the shared project, select a chapter and upload an allowed file. Confirm that its metadata appears in Submissions; then read the supervisor's feedback, grade and comments. Submit a revision where changes are requested.
- v. Use the chapter breakdown to distinguish Approved work from In review or Changes requested. Use tickets for questions/tasks and the activity log for previous topic, review and extension decisions.
- vi. Request a later deadline when necessary and read the decision. After all five chapters are approved, submit the final document and await explicit final approval. Check the separately recorded defence date.
- vii. Check notifications, including a partner-addition notice if another student linked the account to a project. Log out and confirm that the landing page shows sign-in controls and that protected pages require login.

4.5.2 Supervisor

- i. Register or sign in to the Supervisor account using its email and password. Review the dashboard, assigned projects and matriculation-number roster.
- ii. Open a submitted topic and use Approve topic, Decline or Mark pending. Include a decision note where useful. An alternative project-creation form can link an existing departmental student by matriculation number.
- iii. Open each submission, obtain the authorised download, review the current version and save a review state, feedback and an optional grade from 0 to 100. Use submission comments for further discussion.
- iv. Monitor chapter stages, progress, activity and tickets. Review extension requests and grant or decline them. Set the next deadline and the defence date separately.
- v. When all five chapters are approved, review the final submission and grant final approval only if the academic requirements are satisfied. If further chapter changes are needed, return the final review to a non-approved state first.

4.5.3 Head of Department

- i. Sign in with the HOD email account and confirm that project and supervisor summaries belong to the intended department.

- ii. Review the project-health distribution, project list, supervisor workload, response metric and daily activity chart. Open authorised detail views for follow-up.
- iii. Use monitoring information to support departmental discussions; topic decisions, submission grades, extension decisions and final approval remain the assigned supervisor's actions in this version.

4.6 Chapter Summary

This chapter has presented the implemented application, its role-specific interfaces and its completed verification results. Registration, supervisor selection, topic review, shared membership, chapter work, feedback, grading, extensions, final approval and defence-date tracking form a coordinated supervision lifecycle. Chapter progress and project health provide distinct, transparent monitoring outputs. The supplied interface figures illustrate the implemented workspaces, while the test results support the operation of the specified workflow.

REFERENCES

References cited in the report are listed below. The publication details and links follow the corrected literature review.

- Abiola, O. A., Saheed, O. O., & Akinola, S. O. (2024). Student project management system for monitoring undergraduate projects (Case study of Computer Science Department, University of Ibadan). *Journal of Applied Teaching and Learning*, 11(2), 119–130. <https://www.researchgate.net/publication/386986425>
- Abubakar, H., & Muhammed, A. (2022). Technology acceptance model (TAM) as a mechanism for predicting internet use for academic purposes among students of Nigeria Certificate in Education. *International Journal of Education, Technology and Science*, 2(2), 164–180. <https://www.researchgate.net/publication/361099367>
- Adeyemi, T. O., & AlOtaibi, N. F. (2025). Designing a feedback-driven decision support system for dynamic student intervention. *arXiv*. <https://arxiv.org/pdf/2508.07107>
- Adu, M. K., Onwuegbuzie, I. U., Akomolede, K. K., & Idris-Tajudeen, R. (2024). Mitigating the challenges of final year projects supervision in Nigeria tertiary institutions for improved quality assurance. *World Journal of Advanced Engineering Technology and Sciences*, 12(2), 179–188. https://wjaets.com/sites/default/files/fulltext_pdf/WJAETS-2024-0286.pdf
- Angeioplastis, A., Konstantakis, M., & Tsimpiris, A. (2026). Reliable or just accurate? A cross-dataset audit of early-warning models under course-level distribution shift. *Computers*, 15(9), Article 572. <https://www.mdpi.com/2073-431X/15/9/572/pdf>
- Castro, E. G. M. (2024). Mobile-based student internship monitoring system using progress tracking algorithm. *International Research Journal on Advanced Science Hub*, 6(8), 204–209. <https://rspsciencehub.com/index.php/journal/article/download/840/714>
- Cavalcanti, A. P., Barbosa, A., Carvalho, R., Freitas, F., Tsai, Y.-S., Gašević, D., & Mello, R. F. (2021). Automatic feedback in online learning environments: A systematic literature review. *Computers and Education: Artificial Intelligence*, 2, Article 100027. <https://doi.org/10.1016/j.caeai.2021.100027>

- Chacko, S. J., Wang, A.-I. A., Perez-Felkner, L., Haiduc, S., Whalley, D., & Liu, X. (2025). CS-Guide: Leveraging LLMs and student reflections to provide frequent, scalable academic monitoring feedback to computer science students. arXiv. <https://arxiv.org/pdf/2512.19866>
- Chandra Mouli, K., Paramesh, P. M., Niharika, T., et al. (2025). Project tracker management tool. *International Journal on Science and Technology (IJSAT)*, 16(2). <https://www.ijstat.org/papers/2025/2/4299.pdf>
- Davis, F. D. (1989). Perceived usefulness, perceived ease of use, and user acceptance of information technology. *MIS Quarterly*, 13(3), 319–340.
- Gorchakov, A. V., Demidova, L. A., & Sovietov, P. N. (2024). A rule-based algorithm and its specializations for measuring the complexity of software in educational digital environments. *Computers*, 13(3), Article 75. <https://www.mdpi.com/2073-431X/13/3/75/pdf>
- Haris, N. A., Abdullah Zailani, N., Hasim, N., & Abdul Rahman, F. (2025). An integrated approach to supervising and managing students final year projects in higher institutions. *International Journal of Computational Thinking and Data Science*, 5(1), 19–27. <https://semarakilmu.com.my/journals/index.php/CTDS/article/download/14069/12771/88164>
- Hariyanto, F., Budiman, T., Yulianto, A. B., & Yasin, V. (2025). Designing a web-based information system for monitoring final projects. *International Journal of Engineering, Science and Information Technology*, 5(2), 142–153. <https://ijesty.org/index.php/ijesty/article/download/799/423>
- Isa, R., Othman, S., Ali, A. S., Azizan, N., & Ferguson, J. (2024). Prototype development of final year project management system to monitor student's performance. *Journal of Advanced Research in Applied Sciences and Engineering Technology*, 40(1), 164–173. https://semarakilmu.com.my/journals/index.php/applied_sciences_eng_tech/article/download/3389/3934/41888

- Kuzminykh, I., Nawaz, T., Shenzhang, S., Ghita, B., Raphael, J., & Xiao, H. (2024). Personalised feedback framework for online education programmes using generative AI. arXiv. <https://arxiv.org/pdf/2410.11904>
- Li, G., Chen, L., Tang, C., Švábenský, V., Deguchi, D., Yamashita, T., & Shimada, A. (2025). Single-agent vs. multi-agent LLM strategies for automated student reflection assessment. arXiv. <https://arxiv.org/pdf/2504.05716>
- Qian, K., Cheng, Y., Guan, R., Dai, W., Jin, F., Yang, K., Nawaz, S., Swiecki, Z., Chen, G., Yan, L., & Gašević, D. (2025). Dean of LLM tutors: Exploring comprehensive and automated evaluation of LLM-generated educational feedback via LLM feedback evaluators. arXiv. <https://arxiv.org/pdf/2508.05952>
- Shakil, A., & Denny, P. (2024). Enhancing student engagement in large-scale capstone courses: An experience report. arXiv. <https://arxiv.org/pdf/2404.03120>
- Tavares, T. F., & Soares, L. P. (2025). Automated formative feedback for short-form writing: An LLM-driven approach and adoption analysis. arXiv. <https://arxiv.org/pdf/2509.22734>
- van der Merwe, A., du Toit, T., & Kruger, H. (2020). A framework for computerized progress feedback in tertiary education. *Journal of Computer Science*, 16(3), 391–401. <https://doi.org/10.3844/jcssp.2020.391.401>
- Yaba College of Technology. (n.d.). History. <https://www.yabatech.edu.ng/history.php>
- Zhang, X., Lee, V. C. S., Xu, D., Chen, J., & Obaidat, M. S. (2024). An effective learning management system for revealing student performance attributes. arXiv. <https://arxiv.org/pdf/2403.13822>